

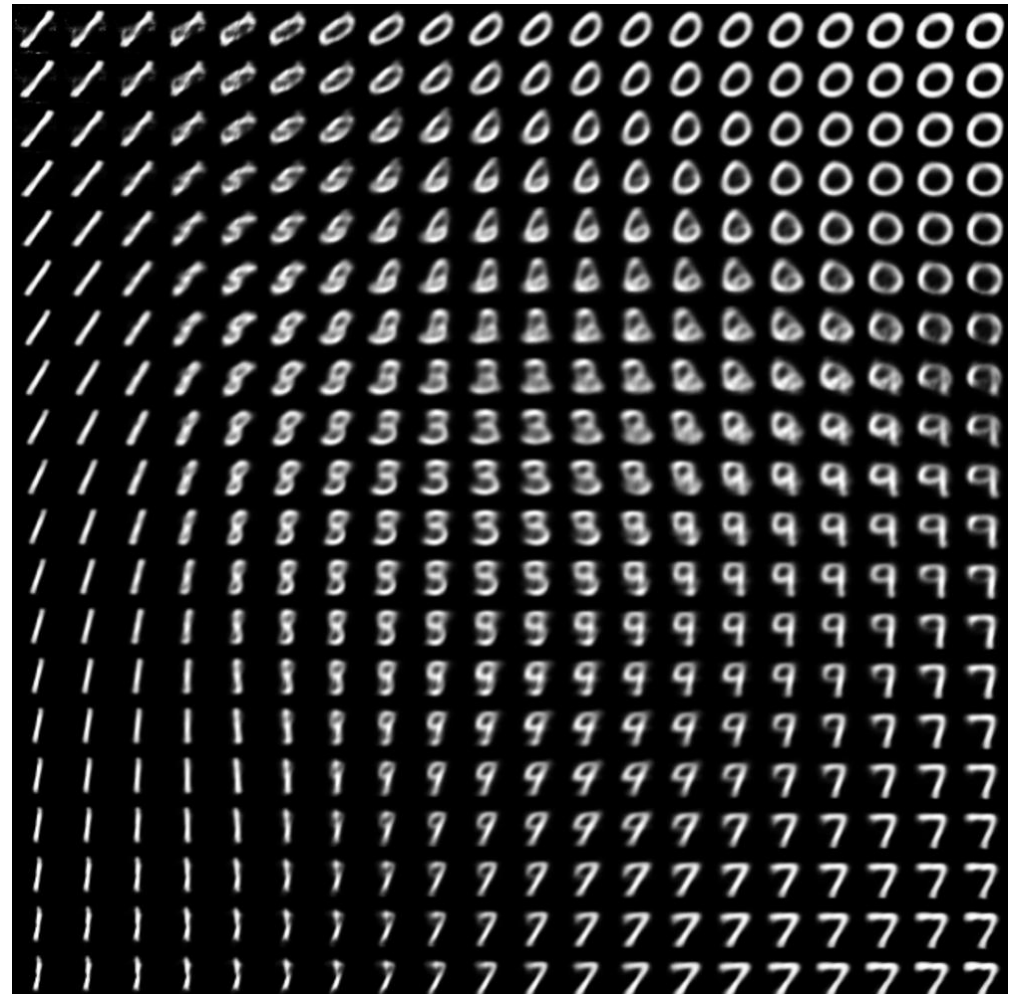
Generative Modeling



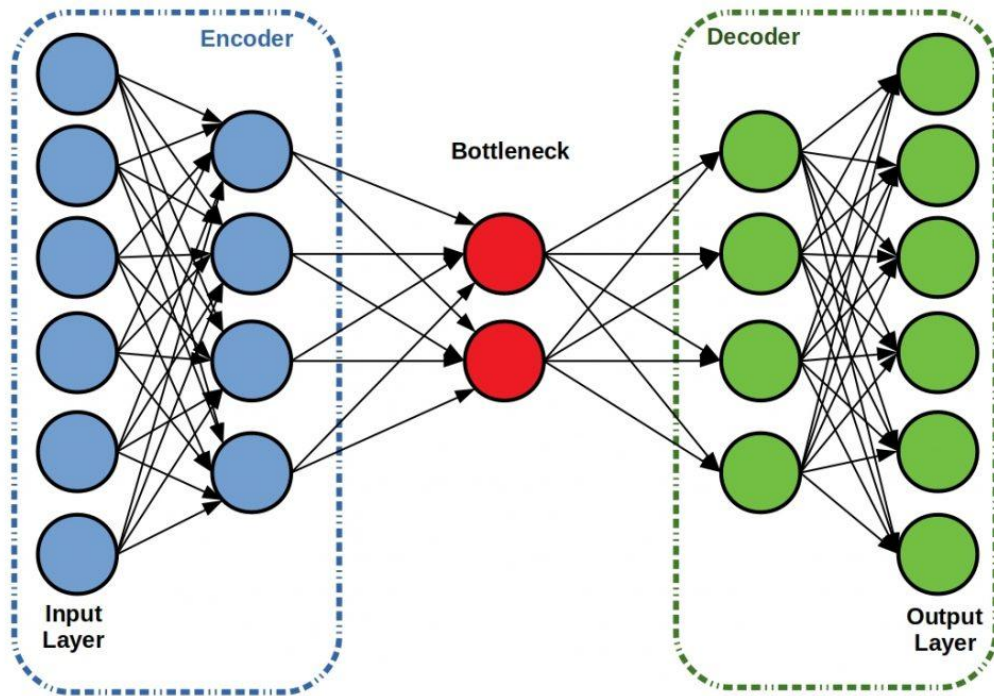
[This Photo](#) is licensed under [CC BY-SA](#).

Agenda

- Autoencoders
- Variational Autoencoders
- Generative Adversarial Networks
- Diffusion models



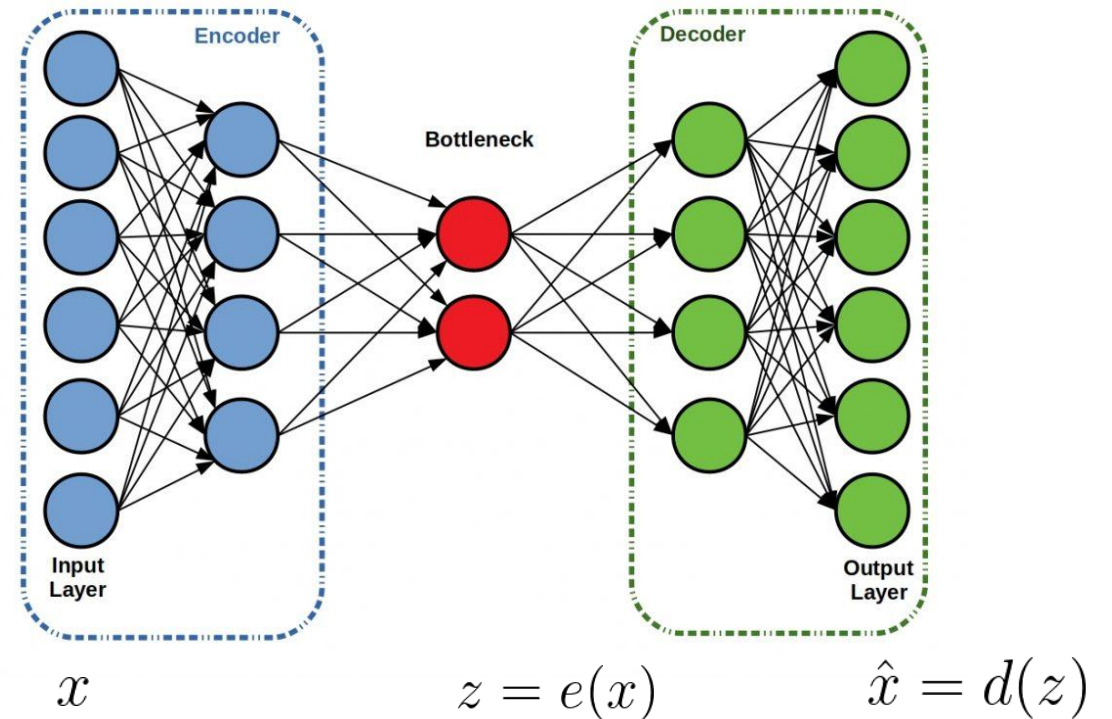
Autoencoder



- Supervised learning problem where the data is its own label!
- Autoencoders are characterized by an encoding and a decoding side, which are connected by a smaller hidden layer called the ***bottleneck***.

Autoencoders

- During **encoding**, the dimension of the input information is reduced.
- In the **decoding** part, the compressed information is used to reconstruct the original data. For this purpose, the weights are then adjusted via backpropagation.
- In the **output layer**, each neuron has the same meaning as the corresponding neuron in the **input layer**

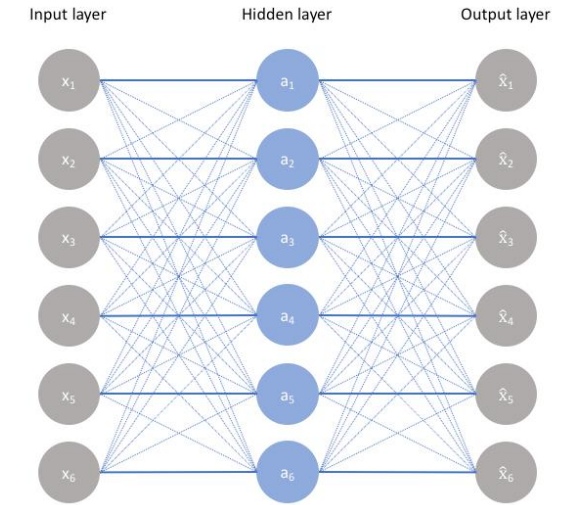


$$Loss = \|x - \hat{x}\|^2$$

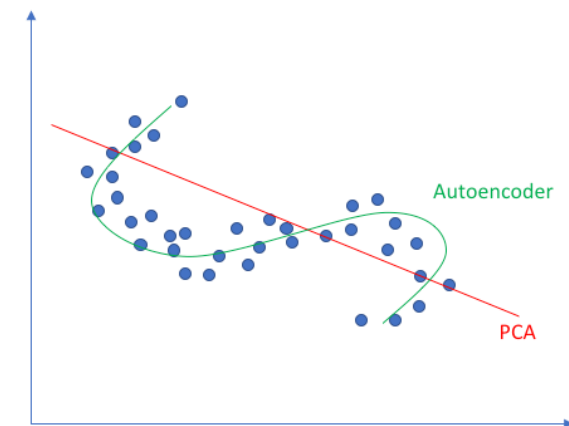
Autoencoder bottleneck

- The bottleneck is essential, since without it, the network could simply learn an identity mapping
- Designing an autoencoder is, therefore, a tradeoff, since it should be able to accurately build a reconstruction, but we don't want it to simply memorize or overfit the training data.
- The encoder does not ensure that the latent space is organized in a smart way for generating new samples (more about this is coming slides).
- Compared to PCA, an autoencoder is more powerful due to non-linearities.

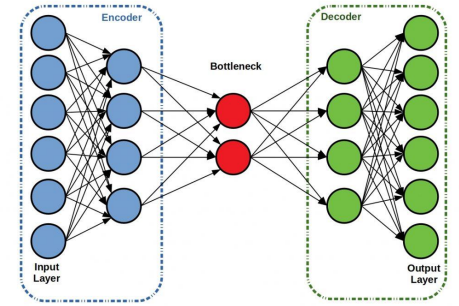
Too wide bottleneck



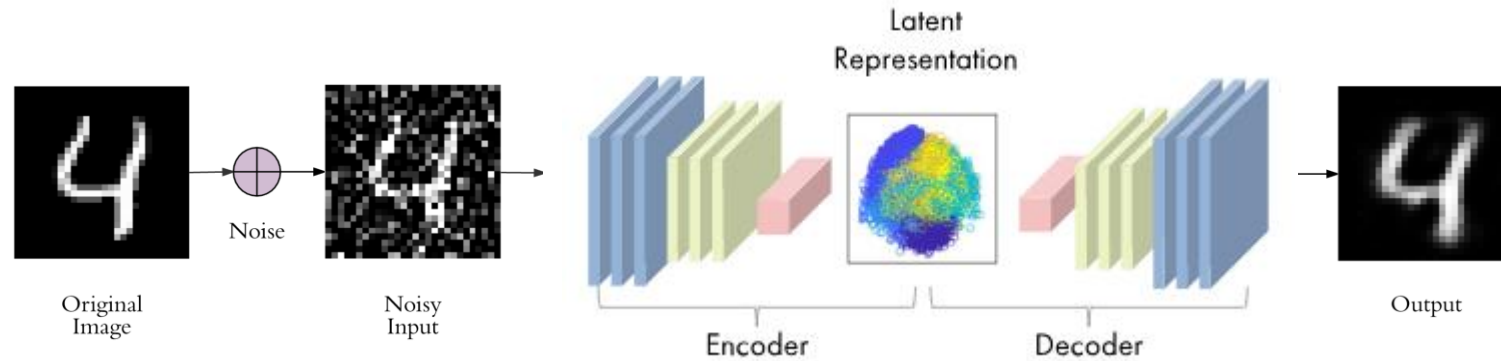
Linear vs nonlinear dimensionality reduction



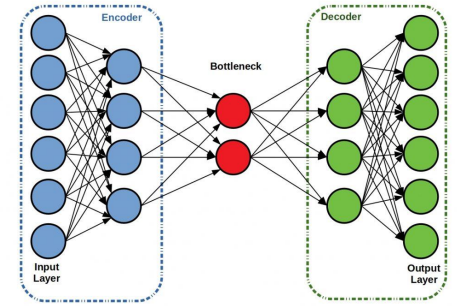
Autoencoders



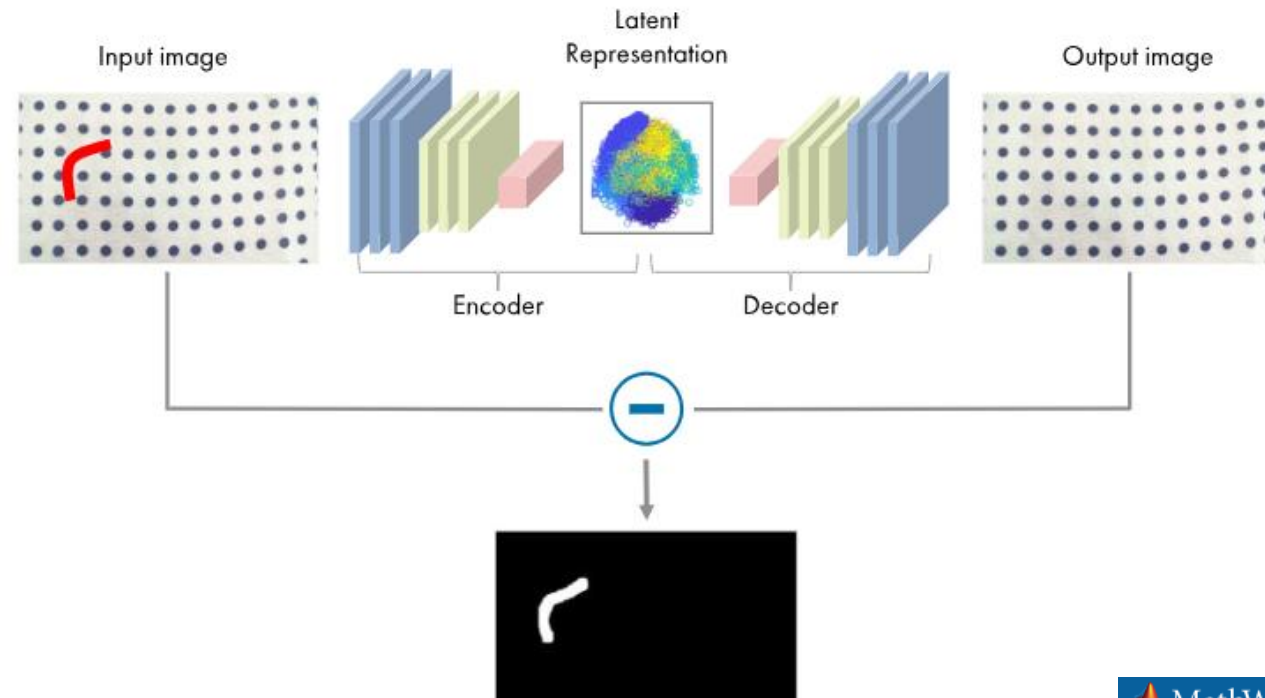
- Typical Autoencoder applications:
 - **Denoising:** Corrupt the input data with noise during training, and use the clean data as label



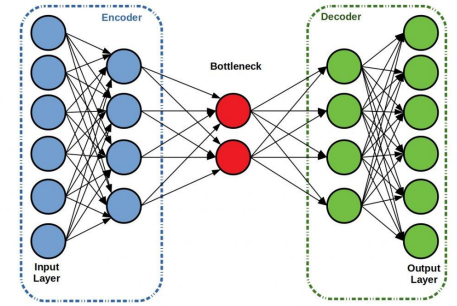
Autoencoders



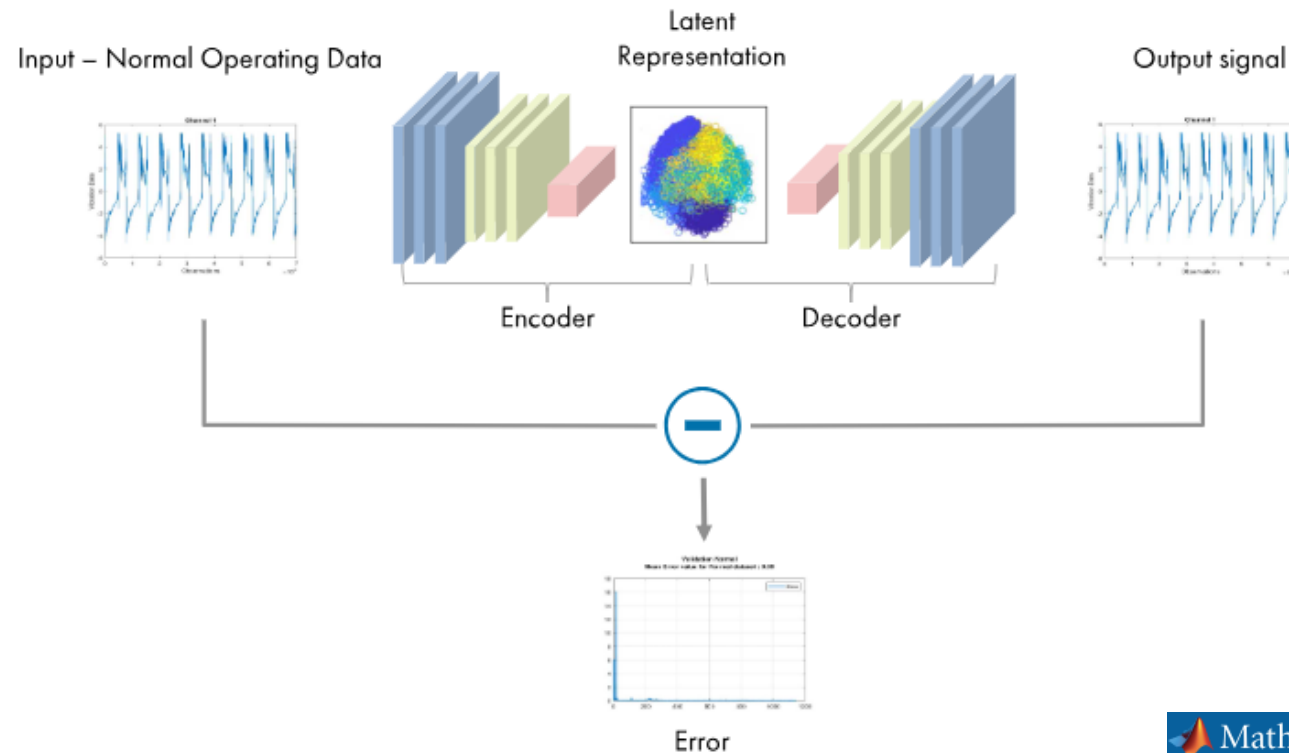
- Typical Autoencoder applications:
 - **Anomaly Detection:** The reconstruction loss of normal samples should be smaller than the reconstruction loss of anomalies



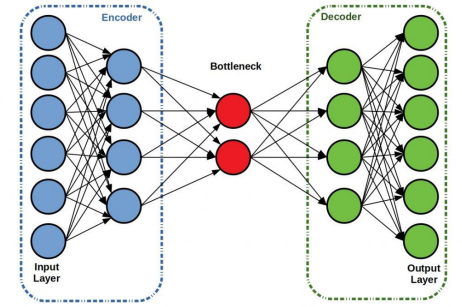
Autoencoders



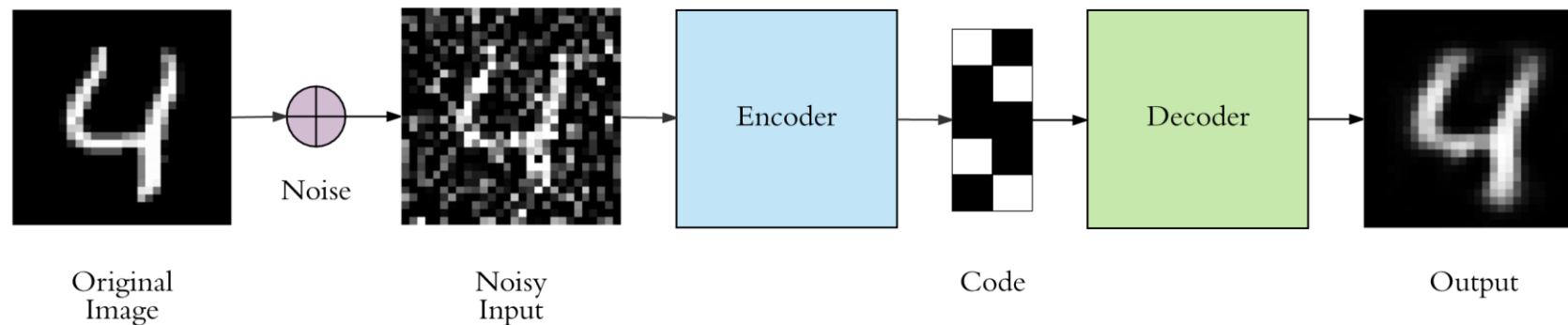
- Typical Autoencoder applications:
 - **Anomaly Detection:** The reconstruction loss of normal samples should be smaller than the reconstruction loss of anomalies



Autoencoders

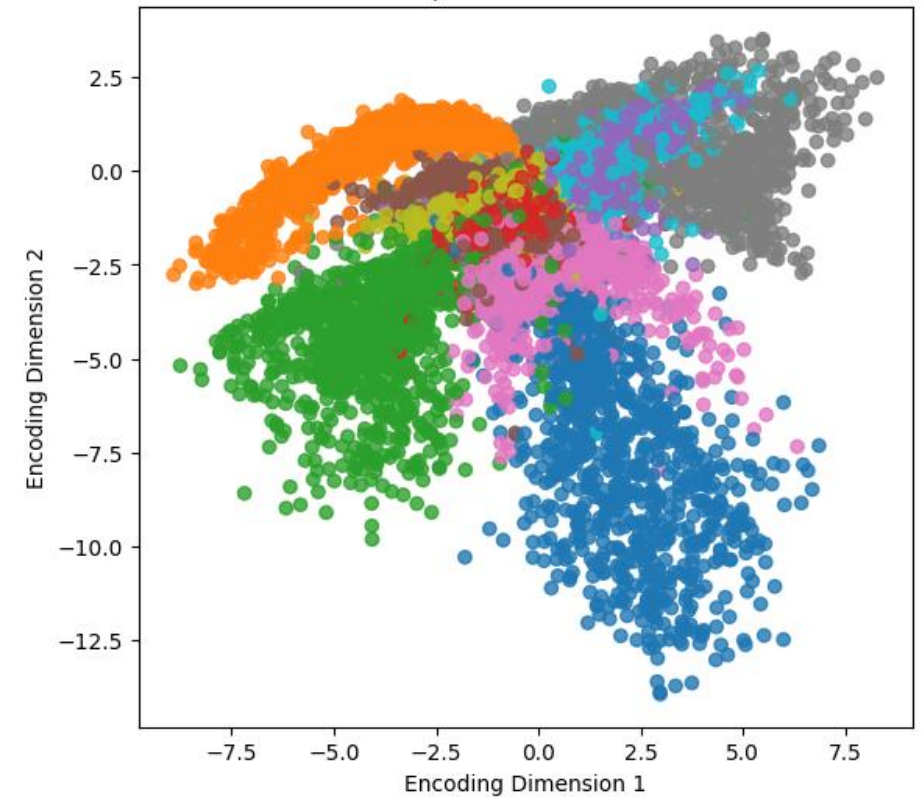


- Typical Autoencoder applications:
 - **Pretraining:** Learn features and reuse the encoder for other tasks, such as classification
 - **Image search:** Make a compressed embedding of images in an image database, compare the encoding of a query image with these embeddings and find images near-by.



Autoencoder latent space (bottleneck)

- This image shows an encoding of MNIST using an MLP autoencoder with a bottleneck sized 2
- The colors represent the MNIST labels
- Notice that most of the labels are entangled! Therefore, we cannot sample from the latent space and expect a meaningful output



Exercise – part 1

- Use the MNIST data and make a simple MLP autoencoder with the following layer where the bottleneck size is 2. The input size will be $28 \times 28 = 784$ neurons at the input.

Layer type	Input size	Output size
linear + relu	# of pixels	100
linear	100	bottleneck_size
linear + relu	bottleneck_size	100
linear	100	# of pixels

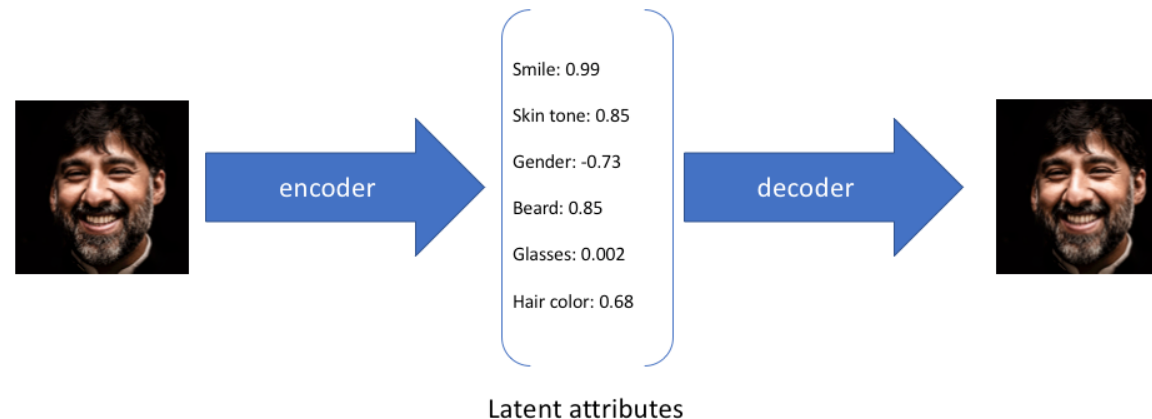
- Show the encoded dimension for the test data set.
- Can you choose some random numbers as encoded dimension and see what output MLP autoencoder generates?



Variational Autoencoder (VAE)

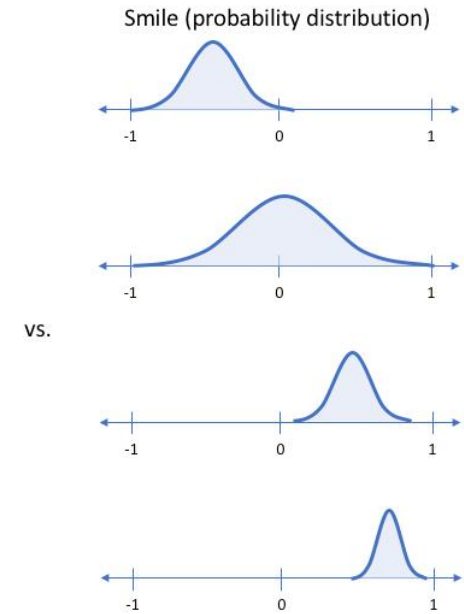
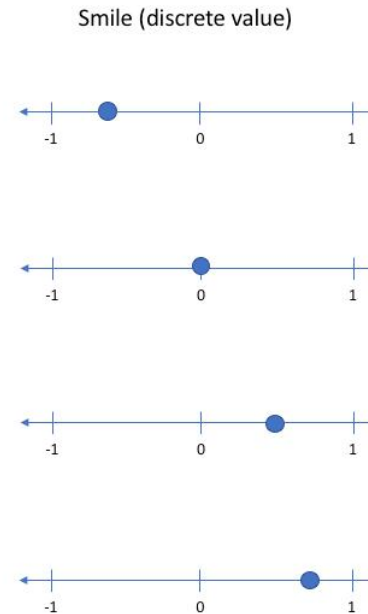
Variational Autoencoder (VAE)

- Learn a representation in the bottleneck, that can be used to generate new samples
- A VAE is an autoencoder whose encoding's distribution is regularized during the training in order to ensure that its latent space has good properties allowing us to generate some new data.



Variational Autoencoder latent space

Rather than representing each latent attribute using discrete values, we encode *each* latent attribute for a given input as a probability distribution over the latent space.



vs.

VAE training procedure

1. The input is encoded as distribution over the latent space. the encoder learns to output two vectors

$$\mu, \sigma$$

which are the mean and variances for the latent vector $\mathbf{z}$, the latent vector $\mathbf{z}$ is then calculated by:

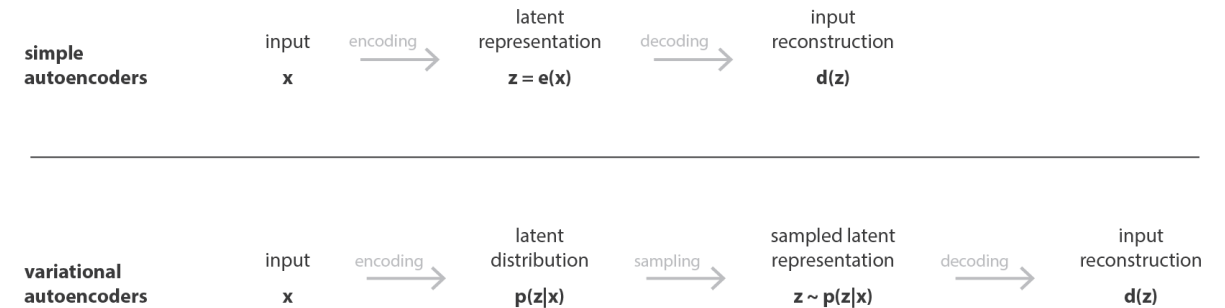
$$\mathbf{z} = \mu + \sigma \epsilon$$

where

$$\epsilon \sim N(0, \mathbf{I}_{z \times z})$$

2. A point from the latent space is sampled from that distribution
3. The sampled point is decoded, and the reconstruction error can be computed
4. The reconstruction error is backpropagated through the network

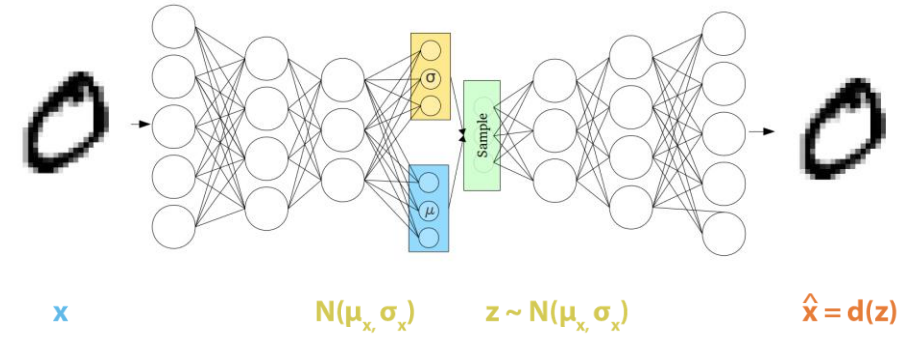
- In practice, the encoded distributions are chosen to be normal so that the encoder can be trained to return the mean and the covariance matrix that describe these Gaussians



VAE Loss function

The loss function is composed of two terms:

- A “reconstruction term” on the final layer.
 - Makes the encoding-decoding scheme as performant as possible.
- A “regularization term” on the latent layer.
 - Regularize the organization of the latent space by making the distributions returned by the encoder close to a standard normal distribution ($\sigma=1$).
 - Expressed as the Kulback-Leibler divergence between the returned distribution and a standard Gaussian, which pushes the returned distribution towards a gaussian distribution
- I.e., we want the output to look like the input, and we want points that are close in the latent space to be similar when decoded.



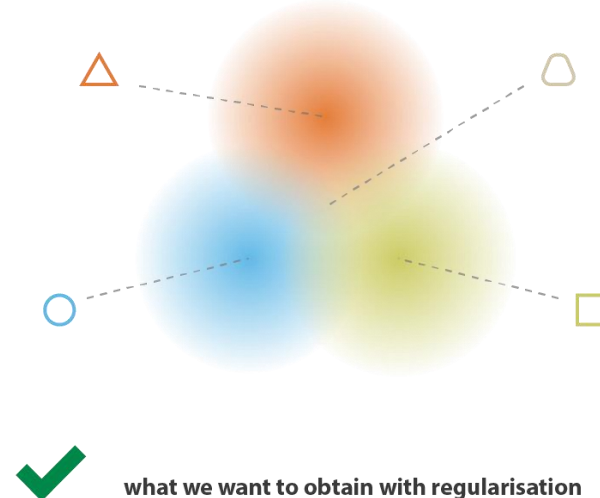
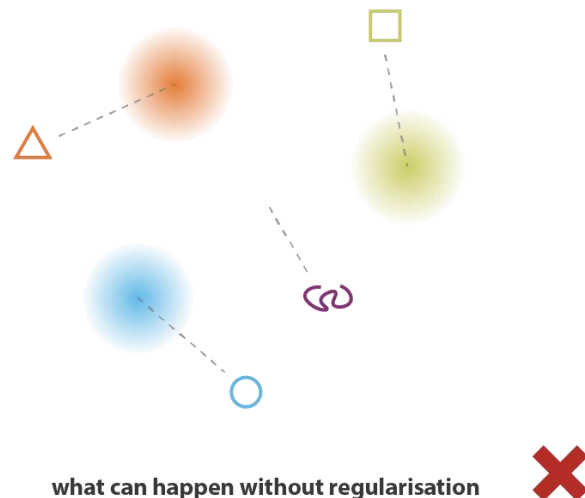
$$\text{loss} = ||x - \hat{x}||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = ||x - d(z)||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)]$$



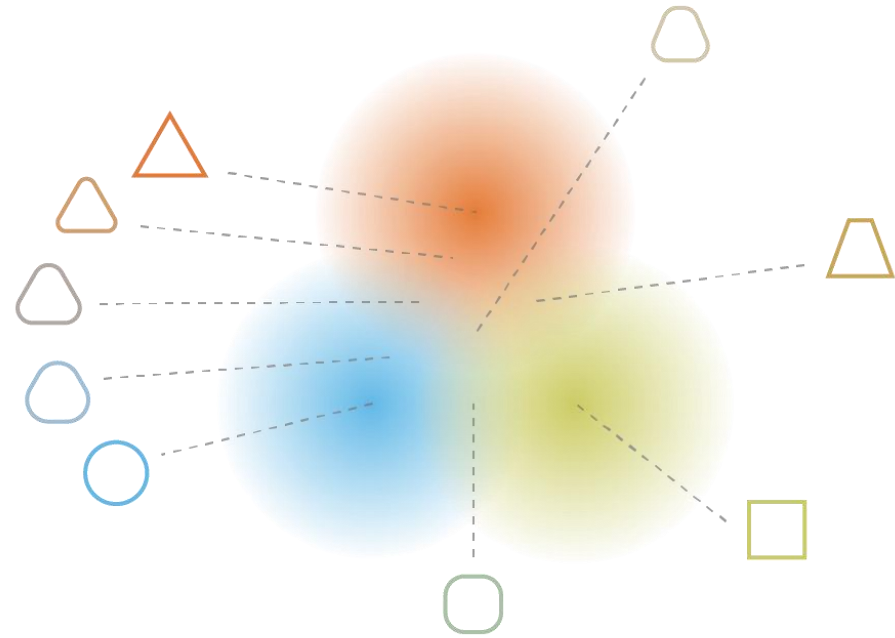
VAE Loss function

- Without a well-defined regularization the encoder can return distributions with tiny variances (~punctual distributions) Or distributions with very different means (that would then be far apart from each other in the latent space).
- The regularization term prevents the model from encoding data far apart in the latent space and encourage as much as possible the returned distributions to “overlap”.

The price of a smooth latent space with distributions that overlap is a higher reconstruction error.



Regularisation tends to create a “gradient” over the information encoded in the latent space.



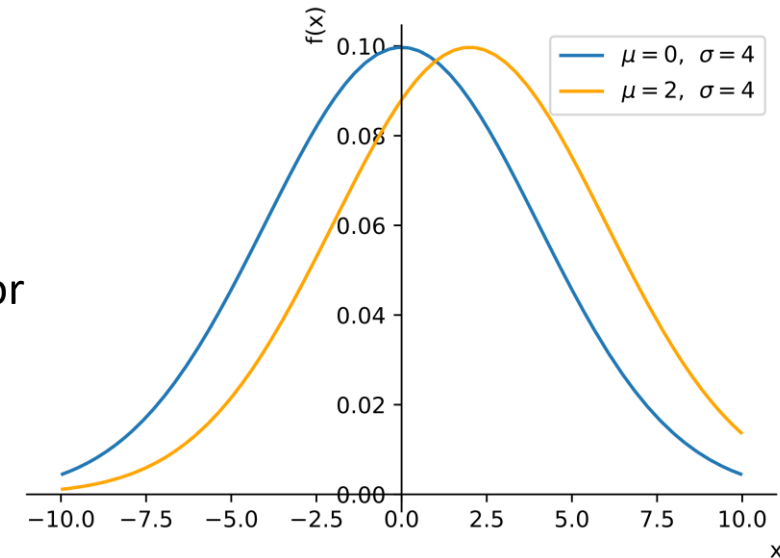
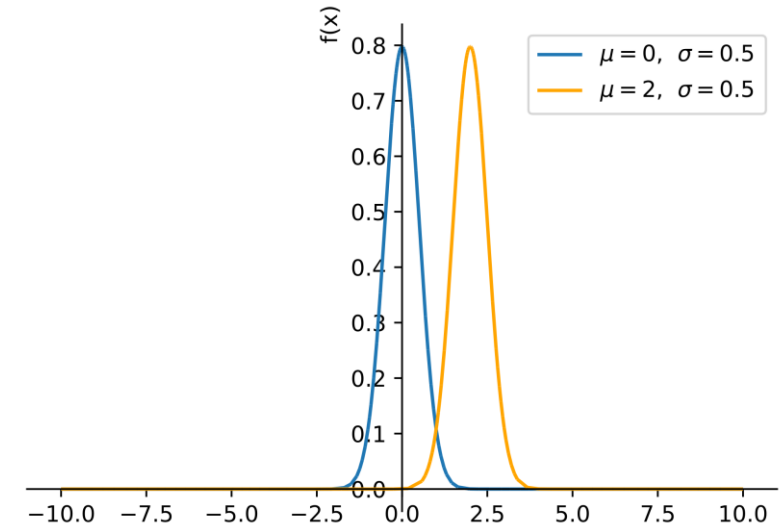
Kullback-Leibler divergence (KL)

A way to measure how one probability distribution is different from a second probability distribution

$$KL(P||Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$
$$KL(P||Q) = \int_x P(x) \log \left(\frac{P(x)}{Q(x)} \right) dx$$

KL Divergence says that if you use distribution P to predict the behavior of some random process, and the process actually happens with distribution Q , how much worse are you doing by using P as an estimate than if I had access to Q directly?

Is called relative distribution in scipy: [scipy.special.rel_entr\(\)](#)



Simply comparing means will not work.

Kullback-Leibler divergence (KL)

Example: comparing two coins: How good is coin2 at predicting the outcome of coin1? If the KL Divergence is 0, then coin2 is a perfect predictor of coin 1

Coin 1: real coin



$\begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$

Coin 2: fake coin



$\begin{cases} q_1 & \text{heads} \\ q_2 & \text{tails} \end{cases}$

Let's make a sequence of trials using coin 1 and calculate the probability that coin 2 generated that sequence. Then we compare that probability to the probability that coin 1 generated the sequence.

Sequence	H	H	T	H	T	H	T	T	H	T	T	H	
Coin 1	p_1	$\cdot p_1$	$\cdot p_2$	$\cdot p_1$	$\cdot p_2$	$\cdot p_1$	$\cdot p_2$	$\cdot p_2$	$\cdot p_1$	$\cdot p_2$	$\cdot p_2$	$\cdot p_1$	$= p_1^{N_h} \cdot p_2^{N_t} = p(\text{observations} \text{coin 1})$
Coin 2	q_1	$\cdot q_1$	$\cdot q_1$	$\cdot q_1$	$\cdot q_2$	$\cdot q_1$	$\cdot q_1$	$\cdot q_2$	$\cdot q_1$	$\cdot q_2$	$\cdot q_1$	$\cdot q_1$	$= q_1^{N_h} \cdot q_2^{N_t} = p(\text{observations} \text{coin 2})$

KL divergence compares the likelihood between these two coins generating the sequence: $KL(P, Q) = \log \left(\frac{p(\text{observations}|\text{coin 1})}{p(\text{observations}|\text{coin 2})} \right)^{\frac{1}{N}}$

KL divergence

Sequence	H	<u>H</u>	T	H	T	H	T	<u>T</u>	H	T	T	H	
Coin 1	p_1	$\cdot p_1$	$\cdot p_2$	$\cdot p_1$	$\cdot p_2$	$\cdot p_1$	$\cdot p_2$	$\cdot p_2$	$\cdot p_1$	$\cdot p_2$	$\cdot p_2$	$\cdot p_1$	$= p_1^{N_h} \cdot p_2^{N_t} = p(\text{observations} \text{coin 1})$
Coin 2	q_1	$\cdot q_1$	$\cdot q_1$	$\cdot q_1$	$\cdot q_2$	$\cdot q_1$	$\cdot q_1$	$\cdot q_2$	$\cdot q_1$	$\cdot q_2$	$\cdot q_1$	$\cdot q_1$	$= q_1^{N_h} \cdot q_2^{N_t} = p(\text{observations} \text{coin 2})$

$$KL(P, Q) = \log \left(\frac{p(\text{observations}|\text{coin 1})}{p(\text{observations}|\text{coin 2})} \right)^{\frac{1}{N}}$$

$$\frac{p(\text{observations}|\text{coin 1})}{p(\text{observations}|\text{coin 2})} = \frac{p_1^{N_h} p_2^{N_t}}{q_1^{N_h} q_2^{N_t}}$$

Normalize by number of samples

$$\Rightarrow \left(\frac{p_1^{N_h} p_2^{N_t}}{q_1^{N_h} q_2^{N_t}} \right)^{\frac{1}{N}}$$

Take the log

$$\Rightarrow \log \left(\frac{p_1^{N_h} p_2^{N_t}}{q_1^{N_h} q_2^{N_t}} \right)^{\frac{1}{N}}$$

Drop down the powers

$$\Rightarrow \frac{N_h}{N} \log p_1 + \frac{N_t}{N} \log p_2 - \frac{N_h}{N} \log q_1 - \frac{N_t}{N} \log q_2$$

Note that $\frac{N_h}{N} = p_1$, $\frac{N_t}{N} = p_2$, when number of observations go to infinity

$$\Rightarrow p_1 \log p_1 + p_2 \log p_2 - p_1 \log q_1 - p_2 \log q_2$$

Same as the original eq! $KL(P||Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$

Kullback-Leibler divergence (KL) derivation

$$D_{KL}(P\|Q) = \int P(x) \log \left(\frac{P(x)}{Q(x)} \right) dx$$

where

$$P(x) = \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp \left(-\frac{(x-\mu_1)^2}{2\sigma_1^2} \right)$$

$$Q(x) = \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{x^2}{2} \right)$$

$$\log \left(\frac{\exp \left(-\frac{(x-\mu_1)^2}{2\sigma_1^2} \right)}{\exp \left(-\frac{x^2}{2} \right)} \right) = -\frac{(x-\mu_1)^2}{2\sigma_1^2} + \frac{x^2}{2}$$

First simplify the log term

$$\log \left(\frac{P(x)}{Q(x)} \right) = \log \left(\frac{\frac{1}{\sqrt{2\pi\sigma_1^2}} \exp \left(-\frac{(x-\mu_1)^2}{2\sigma_1^2} \right)}{\frac{1}{\sqrt{2\pi}} \exp \left(-\frac{x^2}{2} \right)} \right)$$

$$\log \left(\frac{P(x)}{Q(x)} \right) = \underbrace{\log \left(\frac{1}{\sqrt{2\pi\sigma_1^2}} \right)}_{-\frac{1}{2} \log(2\pi\sigma_1^2)} - \underbrace{\log \left(\frac{1}{\sqrt{2\pi}} \right)}_{-\frac{1}{2} \log(2\pi)} + \log \left(\frac{\exp \left(-\frac{(x-\mu_1)^2}{2\sigma_1^2} \right)}{\exp \left(-\frac{x^2}{2} \right)} \right)$$

Finally

$$\log \left(\frac{P(x)}{Q(x)} \right) = -\frac{1}{2} \log(\sigma_1^2) - \frac{(x-\mu_1)^2}{2\sigma_1^2} + \frac{x^2}{2}$$

$$\log \left(\frac{P(x)}{Q(x)} \right) = -\frac{1}{2} \log(\sigma_1^2) - \frac{(x - \mu_1)^2}{2\sigma_1^2} + \frac{x^2}{2}$$

Kullback-Leibler divergence (KL) derivation

$$D_{KL}(P\|Q) = \int P(x) \log \left(\frac{P(x)}{Q(x)} \right) dx$$

Put the log term in the above equation

$$D_{KL}(P\|Q) = \int_{-\infty}^{\infty} P(x) \left[-\frac{1}{2} \log(\sigma_1^2) - \frac{(x - \mu_1)^2}{2\sigma_1^2} + \frac{x^2}{2} \right] dx$$

Split in three integrals

$$D_{KL}(P\|Q) = -\frac{1}{2} \log(\sigma_1^2) \underbrace{\int_{-\infty}^{\infty} P(x) dx}_1 - \frac{1}{2\sigma_1^2} \underbrace{\int_{-\infty}^{\infty} P(x)(x - \mu_1)^2 dx}_{\text{This is the expectation of } (x - \mu_1)^2 \text{ under the distribution } P(x), \text{ which simplifies to variance } \sigma_1^2 \text{ of } P(x)} + \frac{1}{2} \underbrace{\int_{-\infty}^{\infty} P(x)x^2 dx}_{\text{This is the expectation of } x^2 \text{ under the distribution } P(x). \text{ which is a normal distribution } N(\mu_1, \sigma_1^2). \text{ And so, expectation is } E[x^2] = \mu_1^2 + \sigma_1^2}$$

This is the expectation of $(x - \mu_1)^2$ under the distribution $P(x)$, which simplifies to variance σ_1^2 of $P(x)$

This is the expectation of x^2 under the distribution $P(x)$. which is a normal distribution $N(\mu_1, \sigma_1^2)$. And so, expectation is $E[x^2] = \mu_1^2 + \sigma_1^2$

Kullback-Leibler divergence (KL) derivation

$$D_{KL}(P\|Q) = \int P(x) \log \left(\frac{P(x)}{Q(x)} \right) dx$$

Combine the results of all three integrals

$$D_{KL}(P\|Q) = -\frac{1}{2} \log(\sigma_1^2) - \frac{1}{2} + \frac{1}{2}(\mu_1^2 + \sigma_1^2)$$

Simplifying

$$D_{KL}(P\|Q) = \frac{1}{2} (\mu_1^2 + \sigma_1^2 - 1 - \log(\sigma_1^2))$$

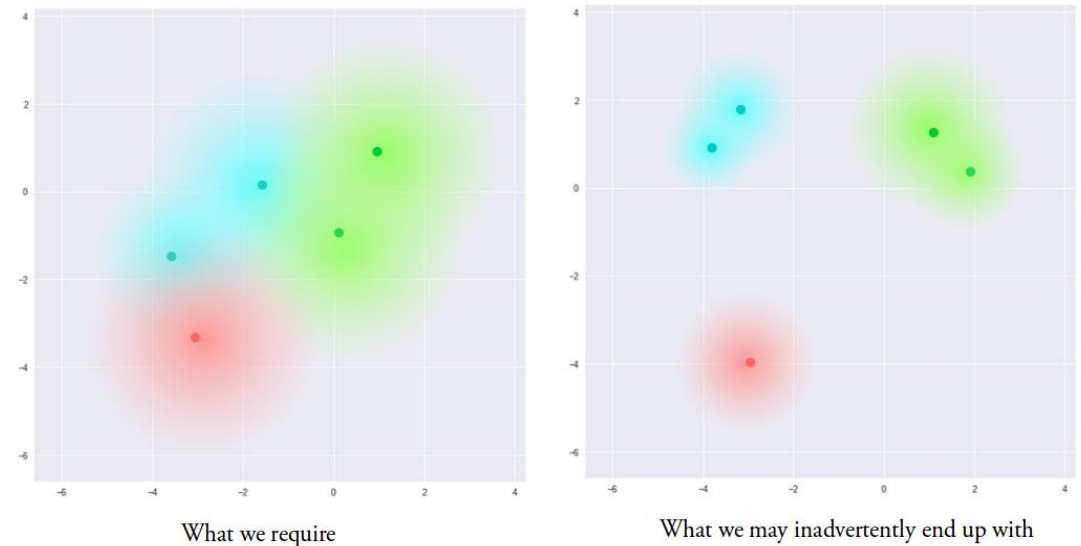
For a case with multiple distributions

$$D_{KL} = \sum_{i=1}^n \sigma_i^2 + \mu_i^2 - \log(\sigma_i) - 1$$

Kullback-Leibler divergence (KL) in VAE

- This loss encourages the encoder to distribute all encodings (for all types of inputs, e.g., all MNIST numbers), evenly around the center of the latent space. If it tries to “cheat” by clustering them apart into specific regions, away from the origin, it will be penalized.
- In the VAE case, the KL divergence can be compressed since $\mu_Q = 0^{\rightarrow}$ and $\Sigma_Q = I$

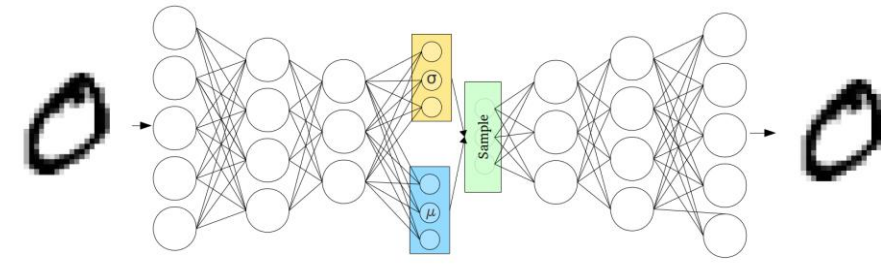
$$D_{KL} = \sum_{i=1}^n \sigma_i^2 + \mu_i^2 - \log(\sigma_i) - 1$$



Check-out: <https://stats.stackexchange.com/questions/318748/deriving-the-kl-divergence-loss-for-vaes/370048#370048>

To see step-by-step, how this expression can be derived from the general KL divergence

Sampling Reparameterization



- z sampled from normal distribution $z \sim \mathcal{N}(\mu(x), \sigma(x))$
 - Sampling z from a distribution is a stochastic process, and this randomness prevents gradients from flowing back through the network during backpropagation.
- Solution: Reparameterization trick
 - Rewrite the sampling process in a way that introduces a deterministic and differentiable transformation, allowing gradients to flow through the network.
 - Instead of sampling z directly from $\mathcal{N}(\mu(x), \sigma(x))$, we express z as:

$$z = \mu(x) + \sigma(x) \cdot \epsilon,$$

where $\epsilon \sim \mathcal{N}(0,1)$

- Randomness is isolated in ϵ , where μ and σ are generated by the neural network and remain differentiable with respect to the input x .

Exercise – part 2

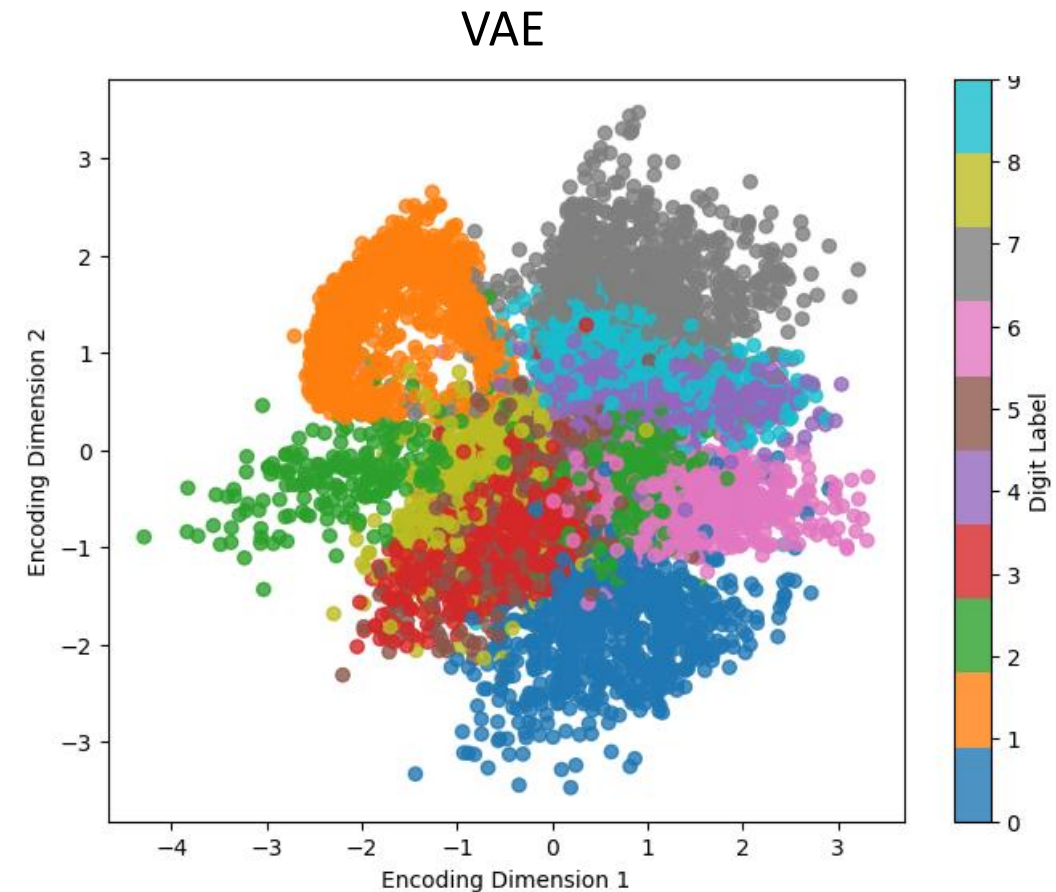
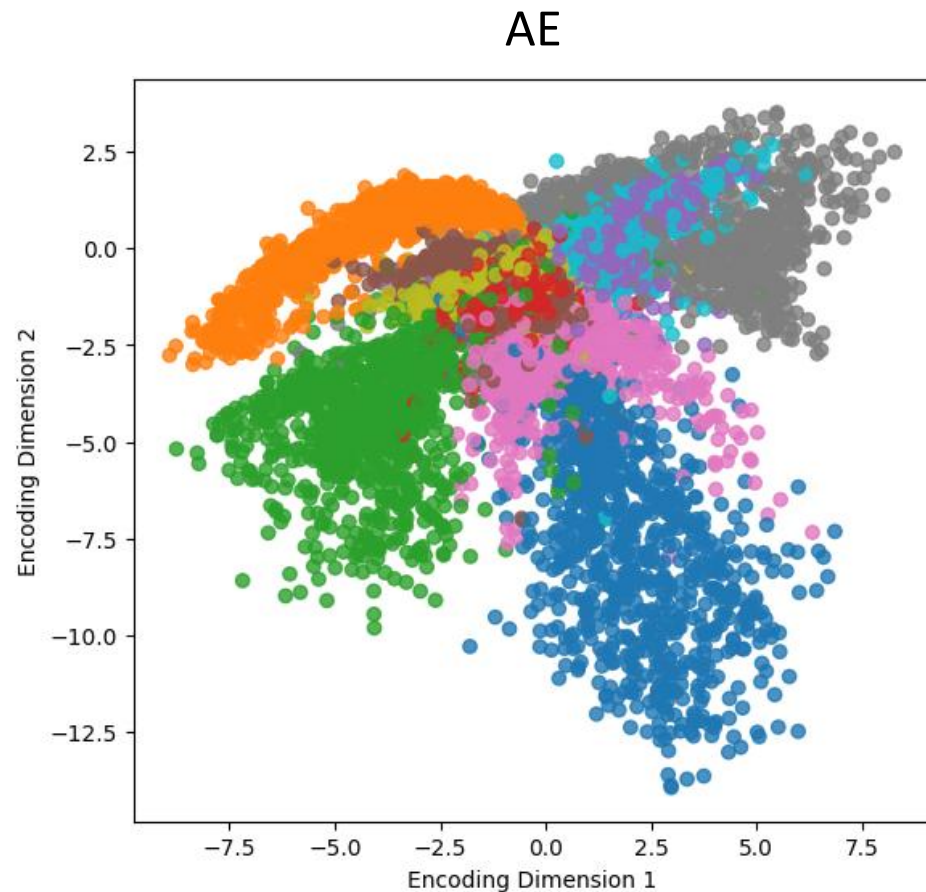
- Use the MNIST data and make a simple MLP autoencoder with the following layer where the bottleneck size is 2. The input size will be $28 \times 28 = 784$ neurons at the input.

Layer type	Input size	Output size
linear + relu	# of pixels	100
linear	100	bottleneck_size
linear + relu	bottleneck_size	100
linear	100	# of pixels

- Show the encoded dimension for the test data set.
- Can you choose some random numbers as encoded dimension and see what output MLP autoencoder generates?
- Modify the same network to make a variational autoencoder. Also show the encoded dimension and choose some numbers for encoded dimension yourself and see what output it generates. Feel free to get help from the internet 😊

VAE latent space (bottleneck)

- Difference between AE and VAE latent space





2014



2015



2016

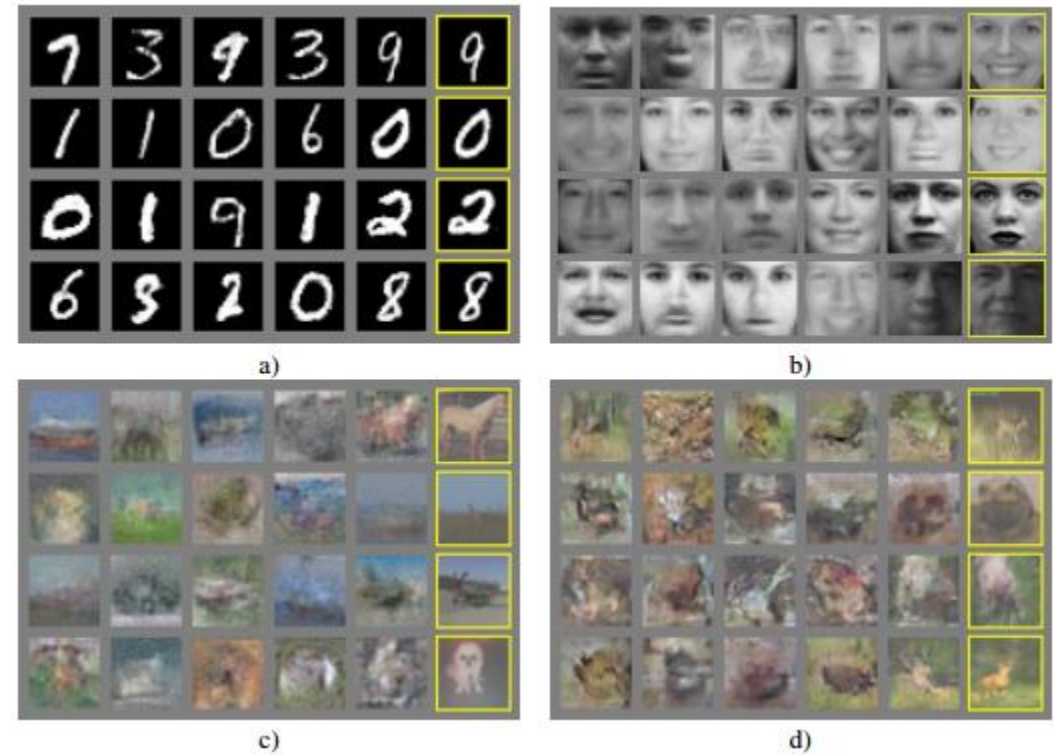


2017

Generative Adversarial Networks

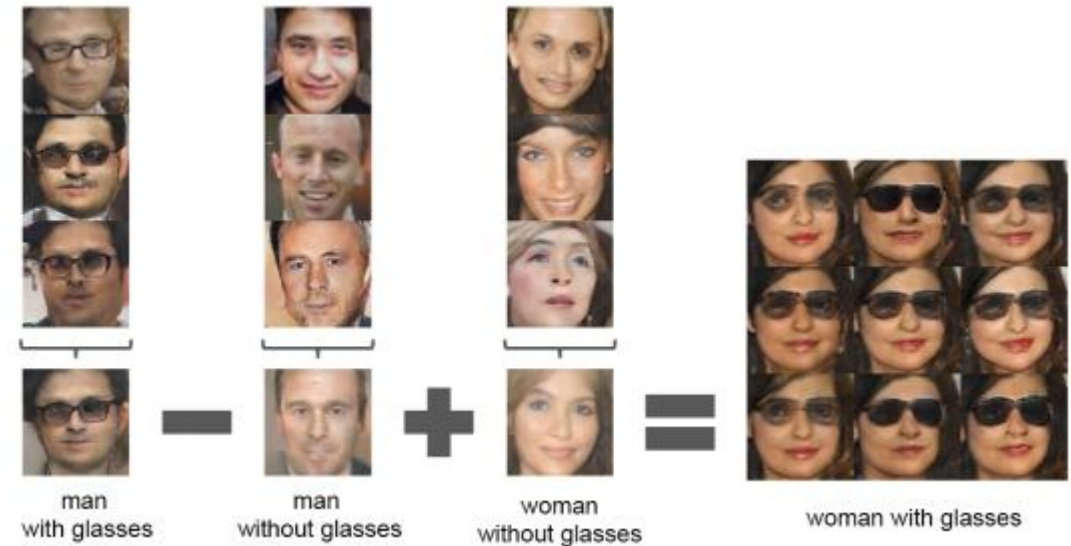
Generative Adversarial Networks (GAN)

- Generated samples and their closest match from Goodfellow et al. 2014
- a), b) and c): Fully connected networks
- d): Convolutional discriminator + “Deconvolutional” generator



Generative Adversarial Network applications

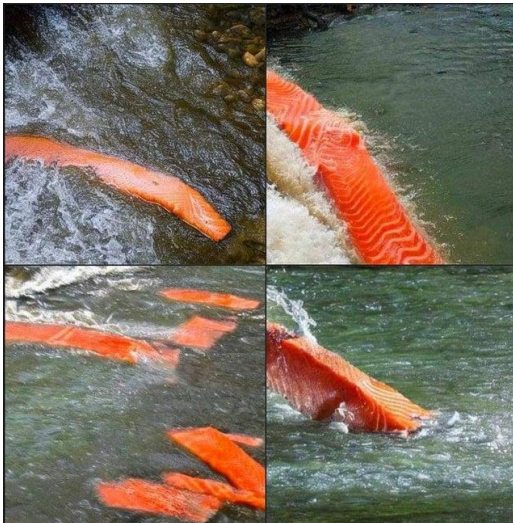
- DCGAN, Radford et al. (2016):
- Arithmetic operations in latent space (z)



Generative Adversarial Network applications

- Text-to-image synthesis with GANs. Image reproduced from Reed et al. (2016b).

Salmon swimming down a river



*on a lighter note

this small bird has a pink breast and crown, and black primaries and secondaries.



the flower has petals that are bright pinkish purple with white stigma



this magnificent fellow is almost all black with a red crest, and white cheek patch



this white and yellow flower have thin white petals and a round yellow stamen



Generative Adversarial Network applications

- StackGAN, Zhang et al. (2016):
- Achieve higher output diversity than other GAN based text-to-image models

This small blue bird has a short pointy beak and brown on its wings



This bird is completely red with black wings and pointy beak



A small sized bird that has a cream belly and a short pointed bill

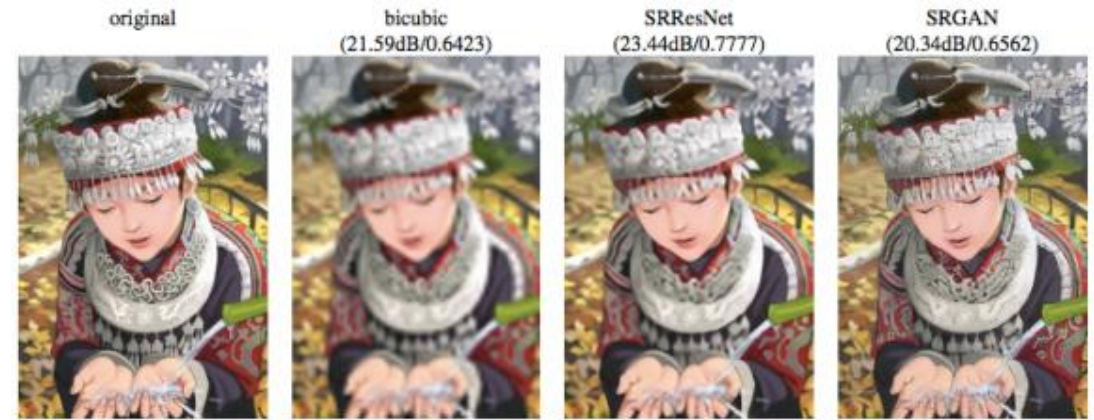


A small bird with a black head and wings and features grey wings



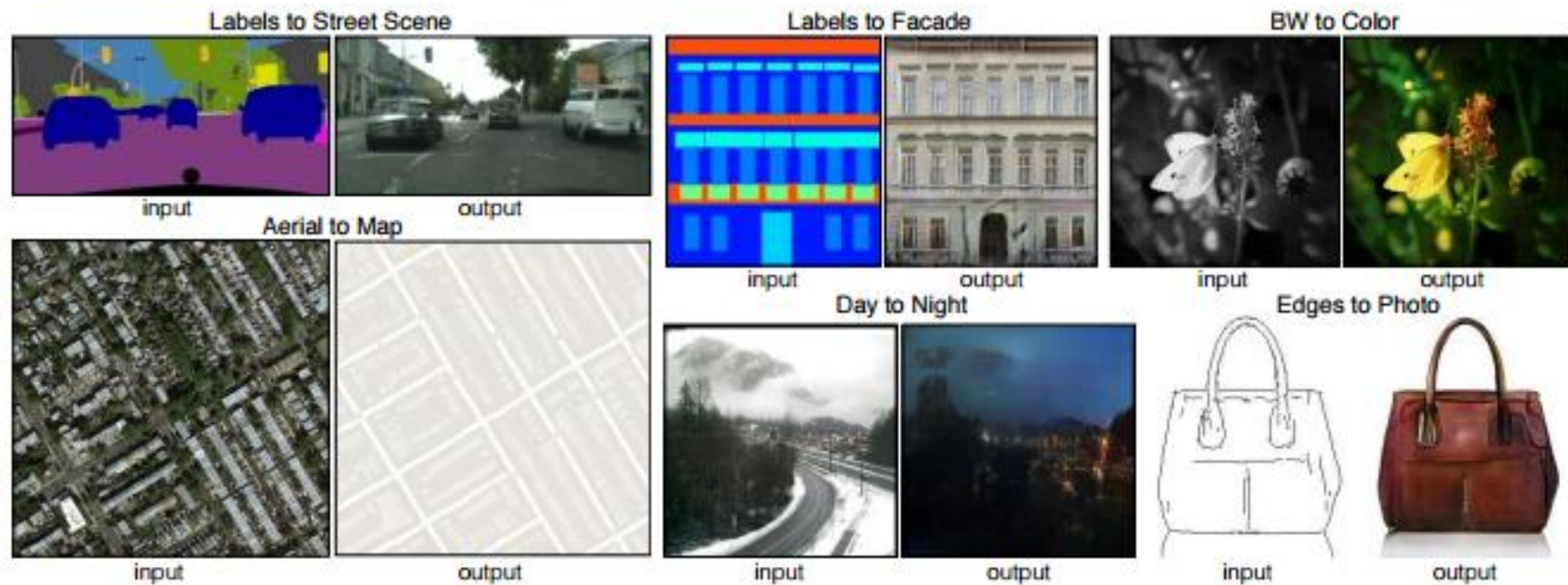
Generative Adversarial Network applications

- SRGAN, Ledig et al. (2016):
- Take a low-resolution image and synthesize a high-resolution equivalent



Generative Adversarial Network applications

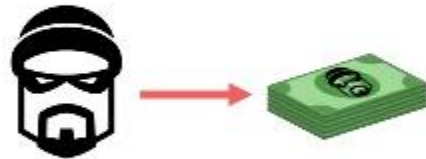
- Image-to-Image Translation with Conditional Adversarial Networks, Isola et al. (2016):



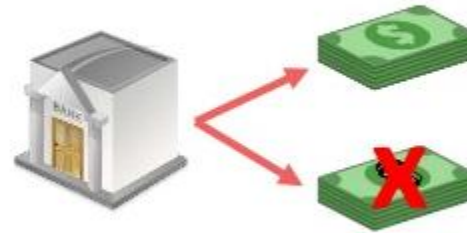
Generative Adversarial Networks (GAN)

What are GANs?

First, an intuition



Goal: produce counterfeit money that is as similar as real money.



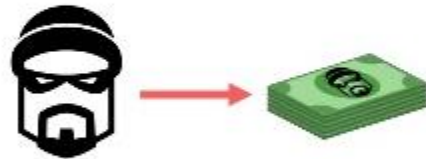
Goal: distinguish between real and counterfeit money.

Generative Adversarial Networks (GAN)

What are GANs?

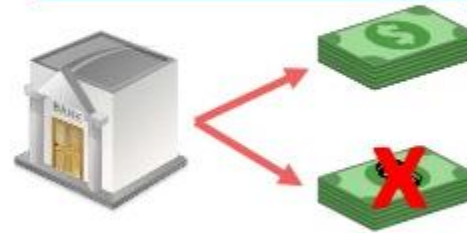
First, an intuition

generator



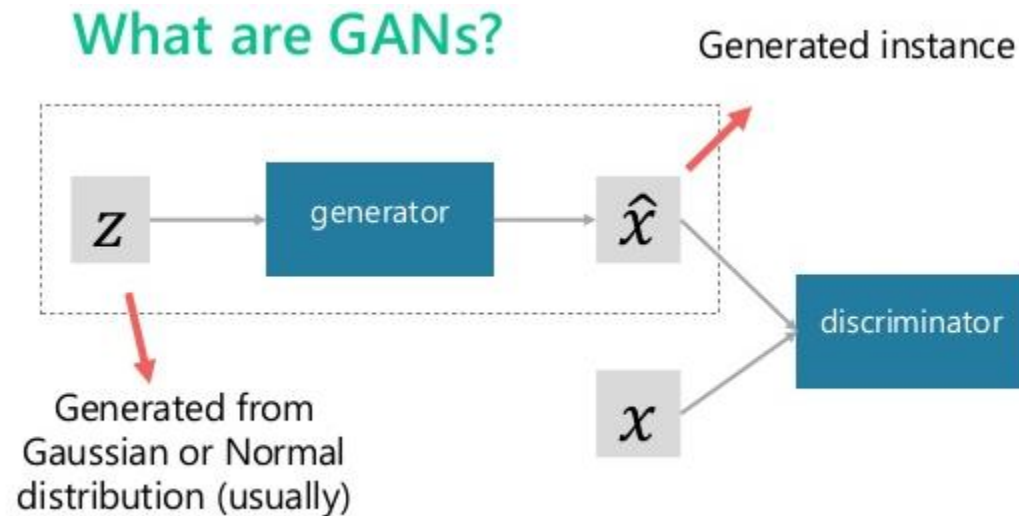
Goal: produce counterfeit money that is as similar as real money.

discriminator



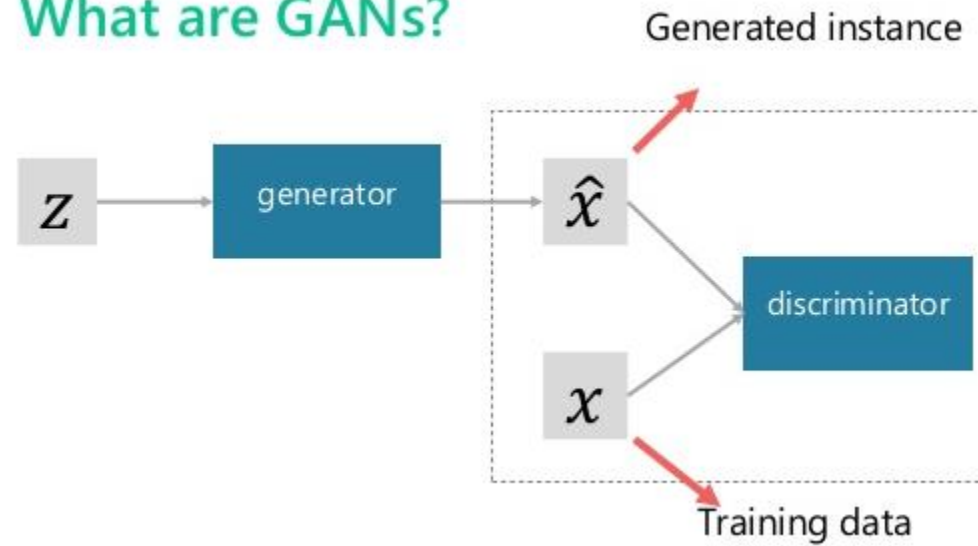
Goal: distinguish between real and counterfeit money.

Generative Adversarial Networks (GAN)



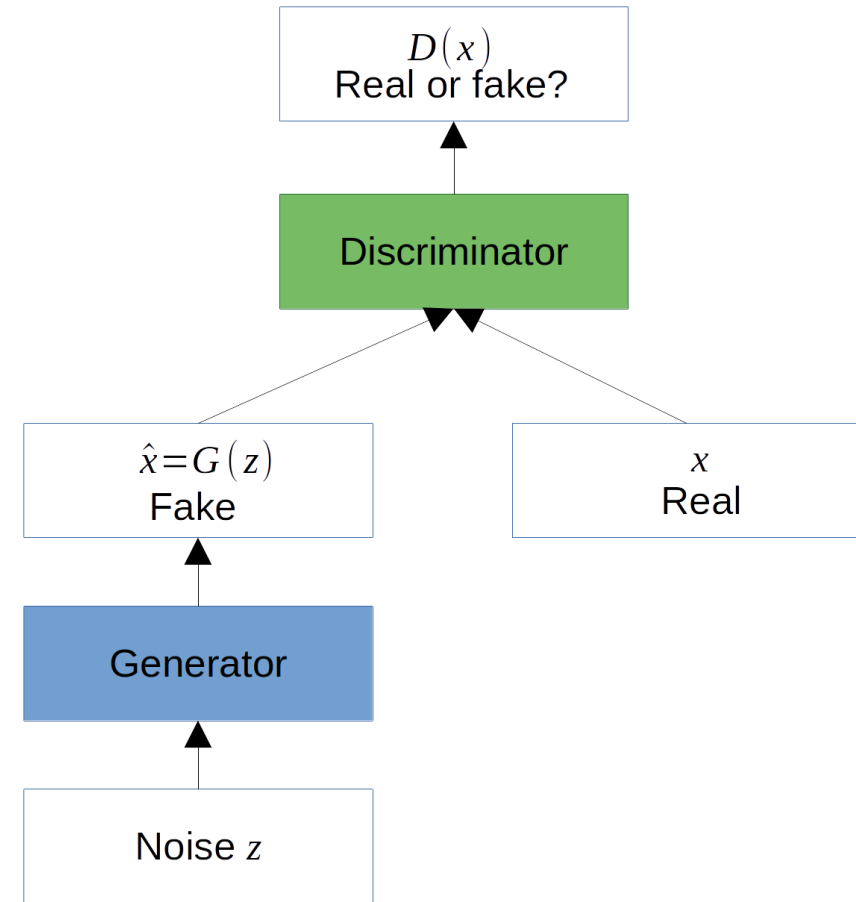
Generative Adversarial Networks (GAN)

What are GANs?



Discriminator

- The discriminator is a classifier that should distinguish real from fake
- Typically, the discriminator outputs a scalar prediction such as using a fully connected layer with size 1 and applies sigmoid function to obtain the predicted probability



$$D(\mathbf{x}) = \frac{1}{1 + e^{-o}}$$

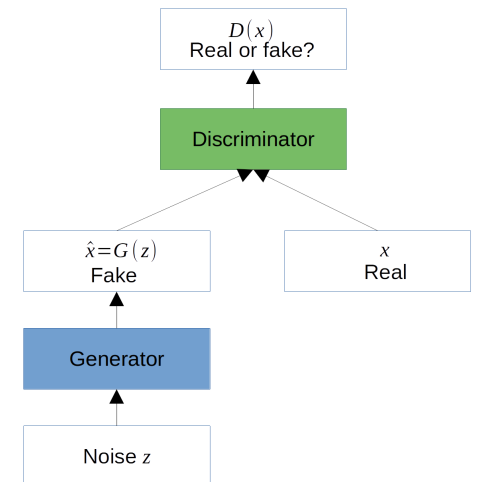
*o = Discriminator output before softmax.
A high score $\rightarrow D(x) = 1$
A low score $\rightarrow D(x) = 0$*

GAN min-max loss

- The standard GAN loss function, also known as the **min-max loss**, was first described in a 2014 paper by Ian Goodfellow et al

$$E_x[\log(D(x))] + E_z[\log(1 - D(G(z)))]$$

- The generator tries to minimize this function while the discriminator tries to maximize it.
 - If G cheats D (i.e., $D(G(z))=1$) the second term becomes $\log(1-1)=-\infty$
 - If D guess G as fake (i.e., $D(G(z))=0$), the second term becomes $\log(1)=0$
 - If D guess x correct, the first term becomes $\log(1)=0$



$D(x)$ is the discriminator's output when given a real data sample x .

$D(G(z))$ is the discriminator's output when given a fake data sample $\hat{x} = G(z)$.

E_x is the expectation over the real data samples, and E_z is the expectation over the noise samples.

Discriminator Loss

- Today, the loss is more commonly implemented as a traditional binary classification problem, where we minimize the cross-entropy loss, i.e.,

$$\min_D \{-y \log D(\mathbf{x}) - (1 - y) \log(1 - D(\mathbf{x}))\}$$

Where the label $y=1$ for the true data and $y=0$ for the fake data.

i.e

True:

$$\min_D \{-\log D(\mathbf{x})\}$$

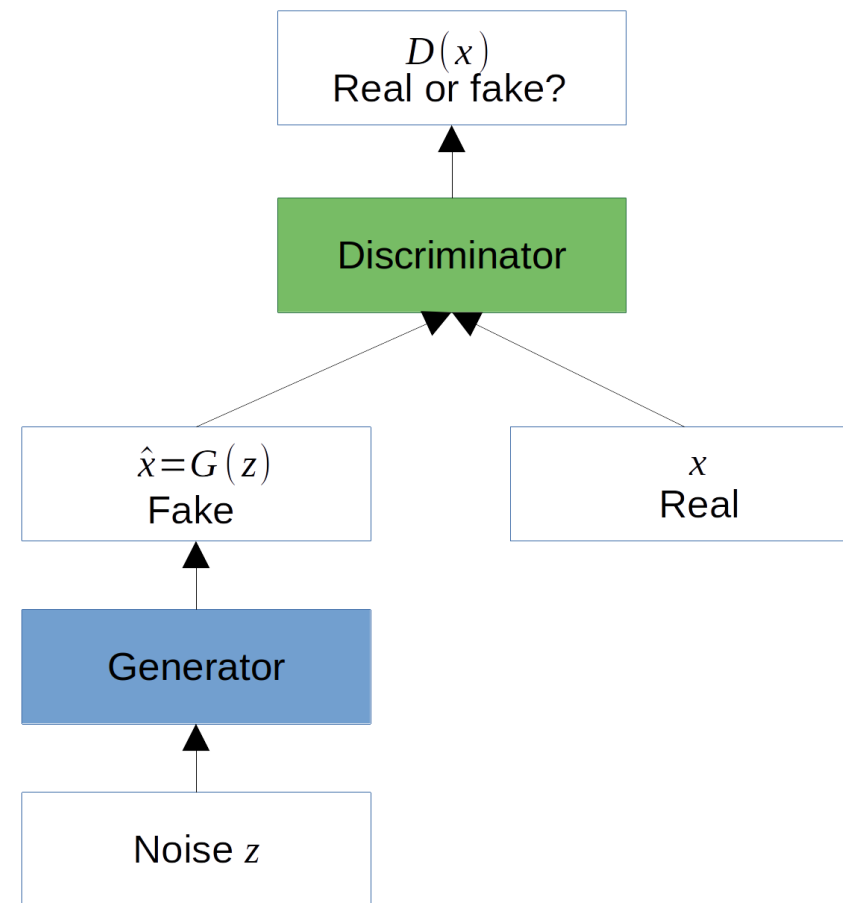
From $-\log(0)=\text{inf}$ to $-\log(1) = 0$
From inf to 0 -> minimization problem

Fake:

$$\min_D \{-\log(1 - D(\mathbf{x}))\}$$

From $-\log(1-1)=\text{inf}$ to $-\log(1-0) = 0$
From inf to 0 -> minimization problem

In practice you will average over batches of fakes and batches of real samples

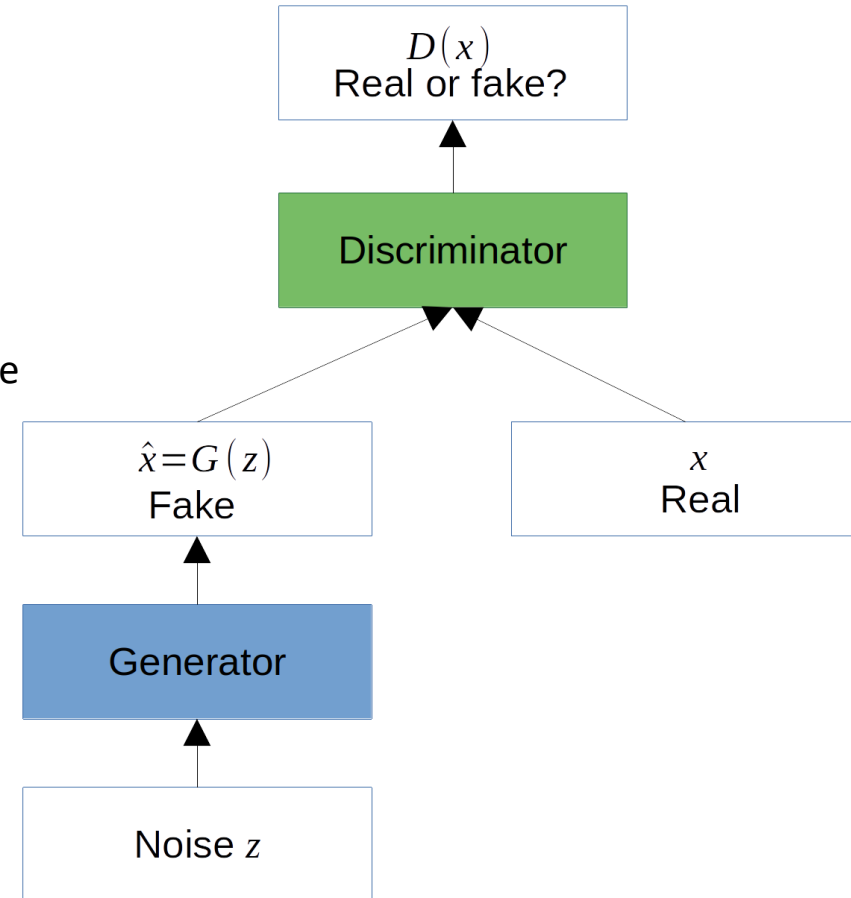


Generator

- Draws some parameter z from a source of randomness. E.g., a normal distribution:

$$\mathbf{z} \sim \mathcal{N}(0, 1) \quad \mathbf{z} \in \mathbb{R}^d$$

- z is often called the latent variable
- The generator applies a function $\hat{x} = G(z)$, that creates a new fake sample.
- This function is often a neural network



Generator Loss

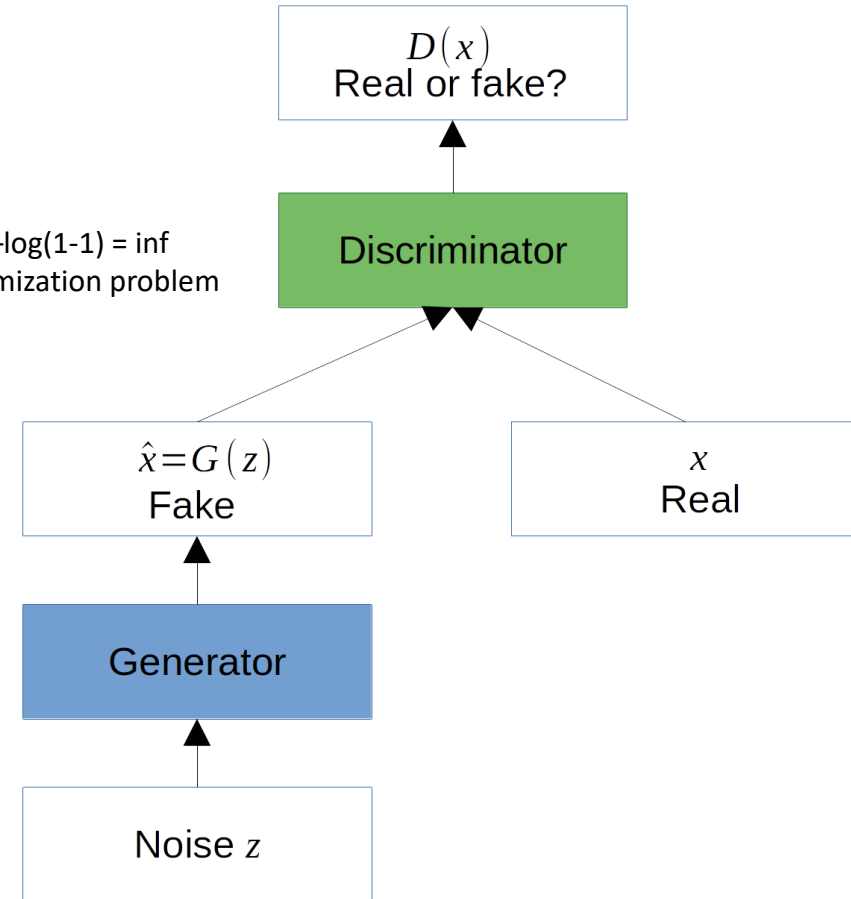
- The goal of the generator is to fool the discriminator to classify $\hat{x} = G(z)$ as true data.
- In other words, for a given discriminator D , we update the parameters of the generator G to maximize the cross-entropy loss when $y = 0$

From $-\log(1-0)=0$ to $-\log(1-1) = \inf$
From 0 to inf -> maximization problem

$$\max_G \{-(1-y) \log(1 - D(G(\mathbf{z})))\} = \max_G \{-\log(1 - D(G(\mathbf{z})))\}$$

- When the generator does a good job at fooling the discriminator, i.e., $D(\hat{x}) \approx 1$, the generator loss drops to zero.
- The generator loss can be transferred to a minimization loss, where we want to minimize the same loss function as the discriminator, but with $y=1$ (true) when the generator outputs :

$$\min_G \{-y \log(D(G(\mathbf{z})))\} = \min_G \{-\log(D(G(\mathbf{z})))\}$$



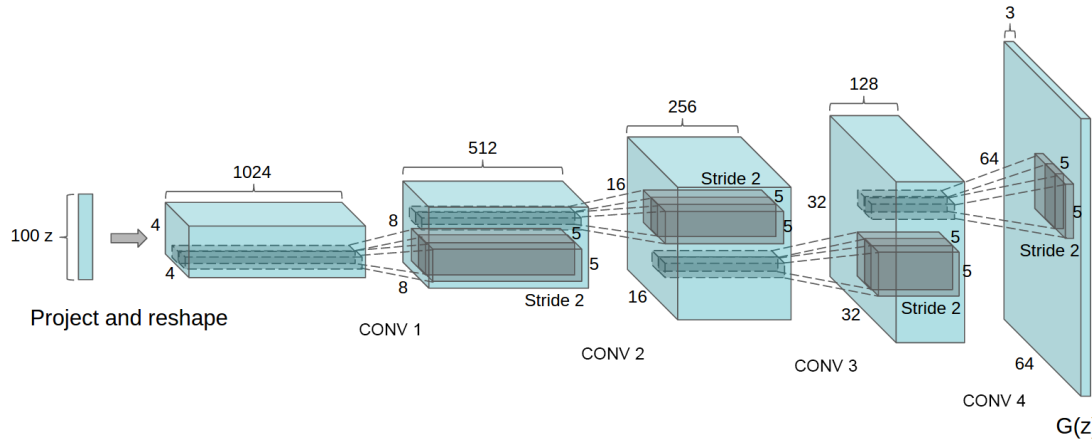
DCGAN (Deep Convolutional GAN)

- Image generators using convolutions
- All convolutions (no fully connected layers)



Generated bedroom images

DCGAN architecture in the org paper: "*Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*"

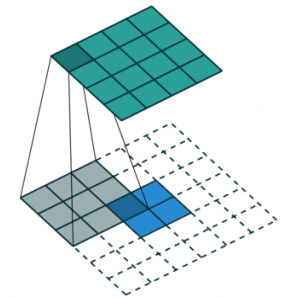


DCGAN generator architecture used for LSUN scene modeling. A 100-dimensional uniform distribution Z is projected to a small spatial extent convolutional representation with many feature maps. A series of four fractionally-strided convolutions (aka transposed conv or deconv) then convert this high-level representation into a 64×64 pixel image.

Notably, no fully connected or pooling layers are used.

Authors architecture guidelines for stable Deep Convolutional GANs

- Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- Use batchnorm in both the generator and the discriminator.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in generator for all layers except for the output, which uses Tanh.
- Use LeakyReLU activation in the discriminator for all layers.



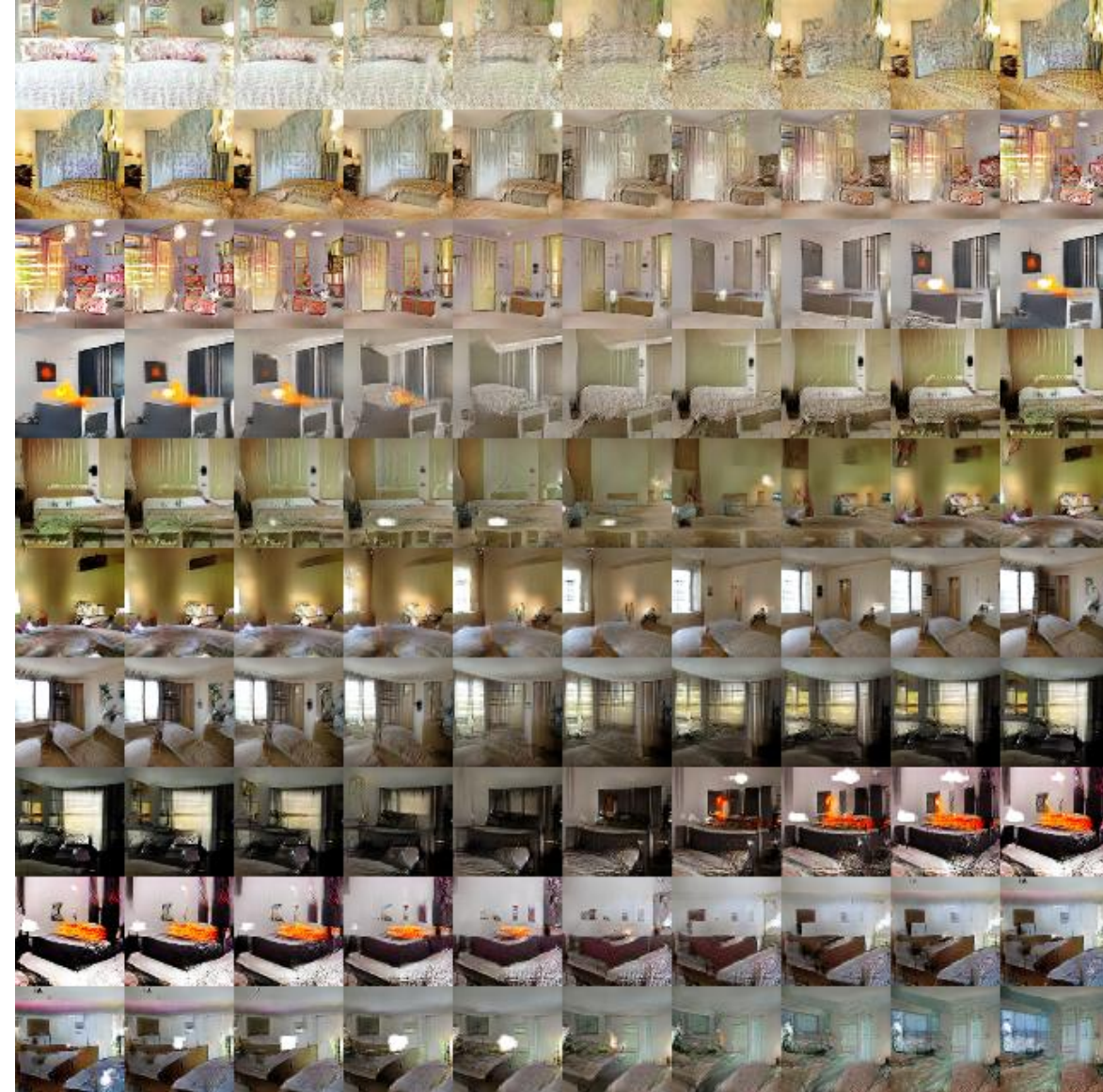
`torch.nn.ConvTranspose2d`

Authors' just state that they found this to work well



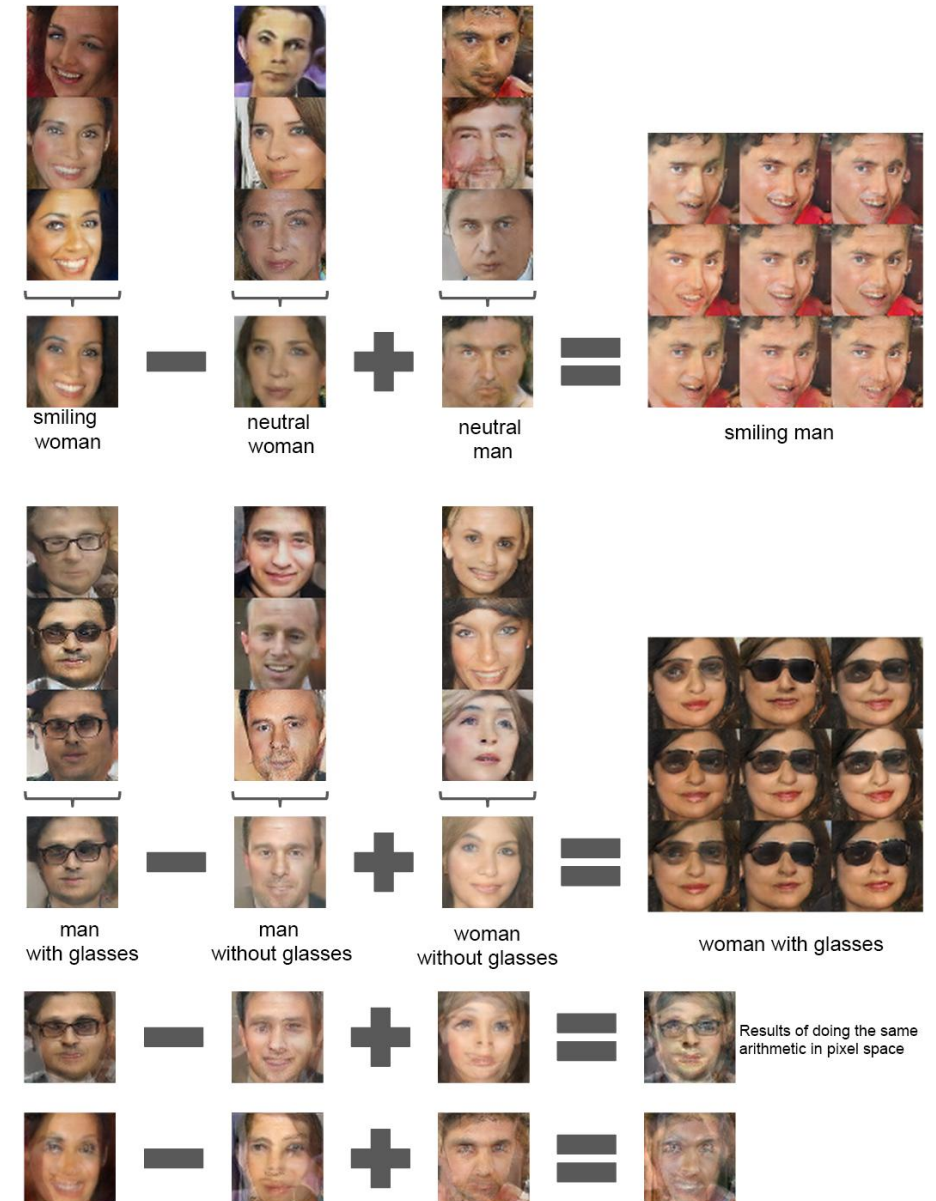
DC GAN: Walking in the Latent space

- Change what the network outputs by walking along the Z input vector.
- (image) Interpolation between a series of 9 random points in Z show that the space learned has smooth transitions



DC GAN: Arithmetic

- Vector arithmetic for visual concepts.
- How to:
 - Find noise vector that represents man with glasses
 - Find noise vector of neutral man
 - Find noise vector of woman without glasses.
 - Subtract and add these three vectors and generate a new sample
 - Woila!



You will be given a DCGAN code, which you will try to understand 😊

Generator mode collapse

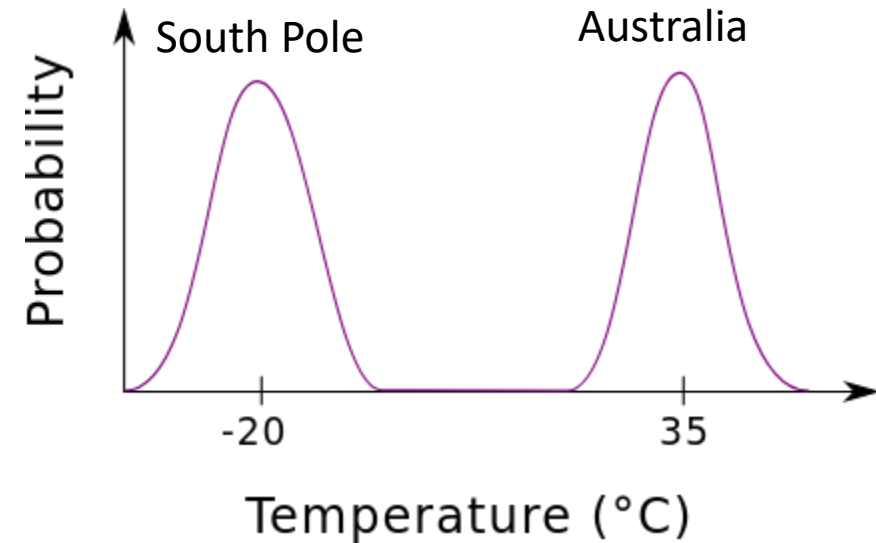
- Mode collapse means that the generator only outputs samples from a single mode
- Usually, we want our generator to be able to produce a range of outputs depending on the input noise, but the generator might learn a distribution that fools the discriminator and keep producing this distribution.
- The discriminator will learn that that distribution is fake, while others are real.
- Problems like mode collapse happen because the generator winning a turn in the game does not correlate with actually reducing the distances between the generated and real-world probability distributions.



Mode collapse example:

A dataset with temperatures from two locations.

1. The generator learns that it can fool the discriminator into thinking that it is outputting realistic temperatures by producing values close to Antarctic temperatures.
2. The discriminator counters by learning that all Australian temperatures are real (not produced by the generator), and essentially guesses whether Antarctic temperatures are real or fake since they are indistinguishable.
3. The generator exploits the discriminator by switching modes to produce values close to Australian temperatures instead, abandoning the Antarctic mode.
4. The discriminator now assumes that all Australian temperatures are fake and Antarctic temperatures are real.
5. Return to step 1.

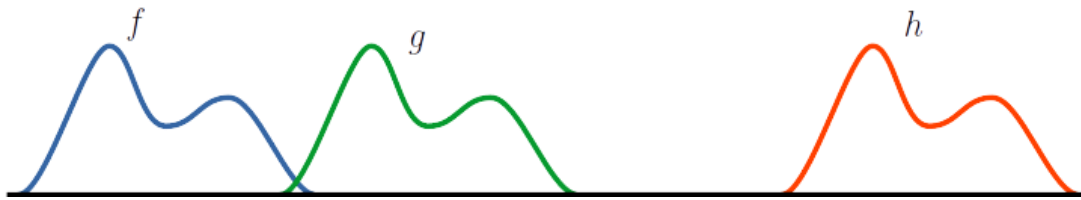


Fighting mode collapse using Wasserstein distance aka Earth mover distance as loss function

- Wasserstein distance can be used as a loss function to make the distribution of generated samples close to the distribution of real samples
- One possible distance measure between these two "piles of earth" is how much work we need to perform to turn the first pile of earth into the other pile.
- We choose the transport plan of one pile of dirt into another with the lowest possible work



Wasserstein distance



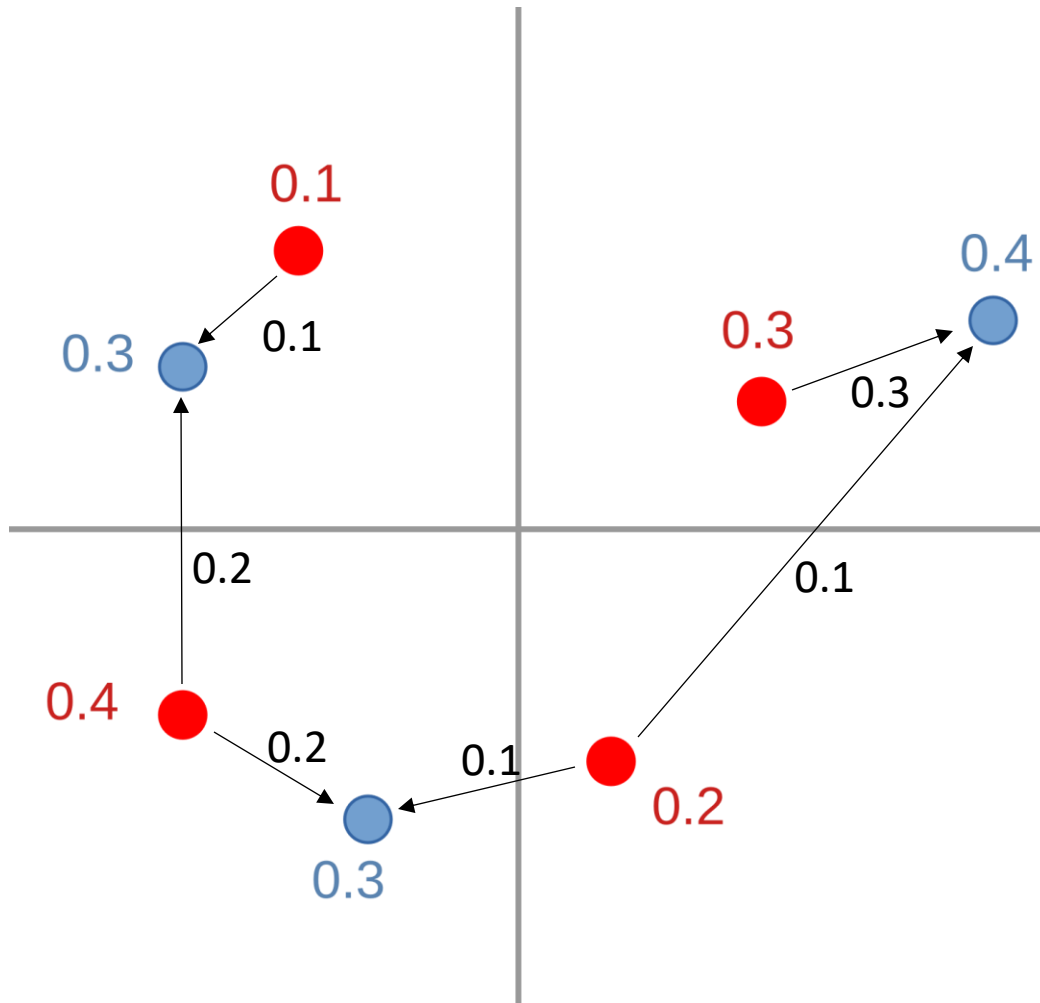
- Consider the three probability functions, f , h , and h
- The functional distance between f and g is almost the same as the functional distance between f and g

$$\|f - g\| \approx \|f - h\|$$

- With Wasserstein

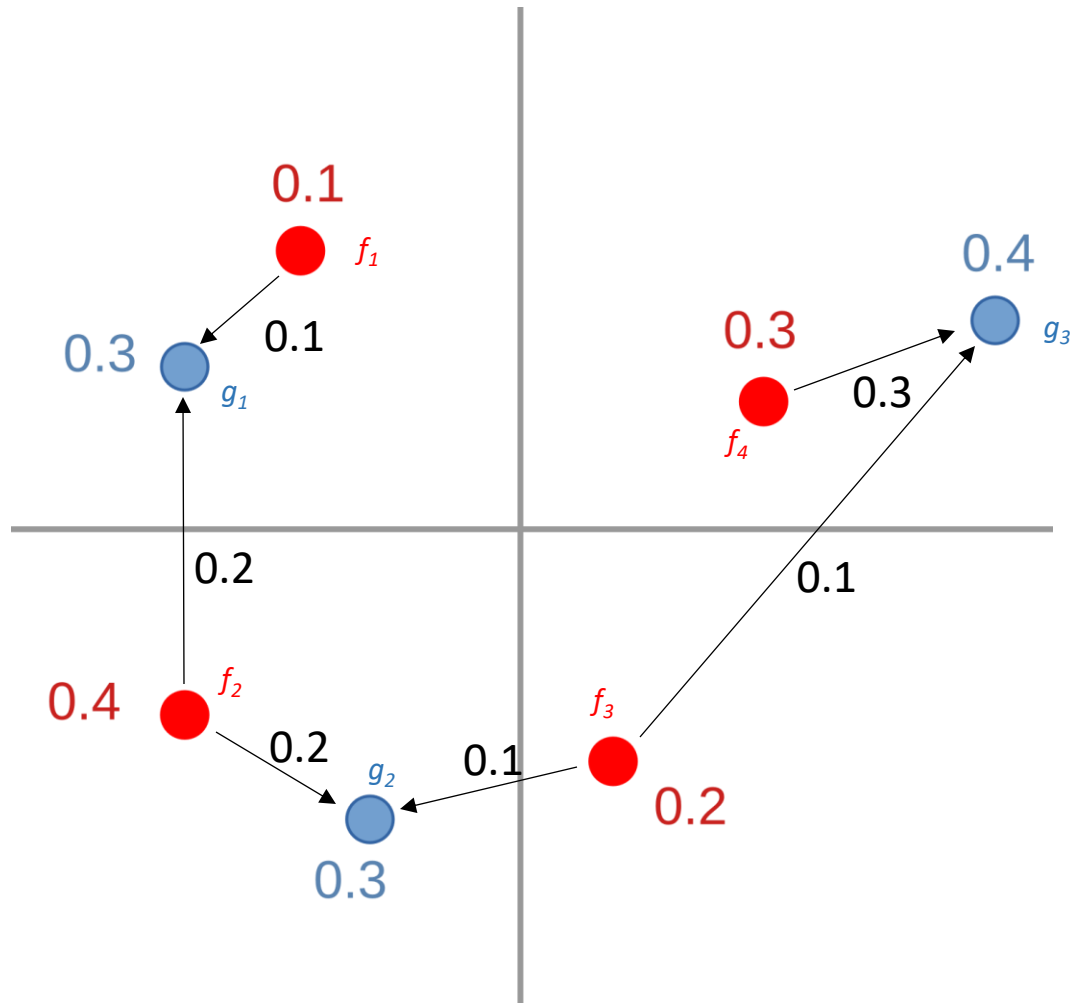
$$d_w(f, g) < d_w(f, h)$$

Wasserstein example



- Two distributions, f and g . We want to move f to g
- *Let's try one strategy:*

Wasserstein example



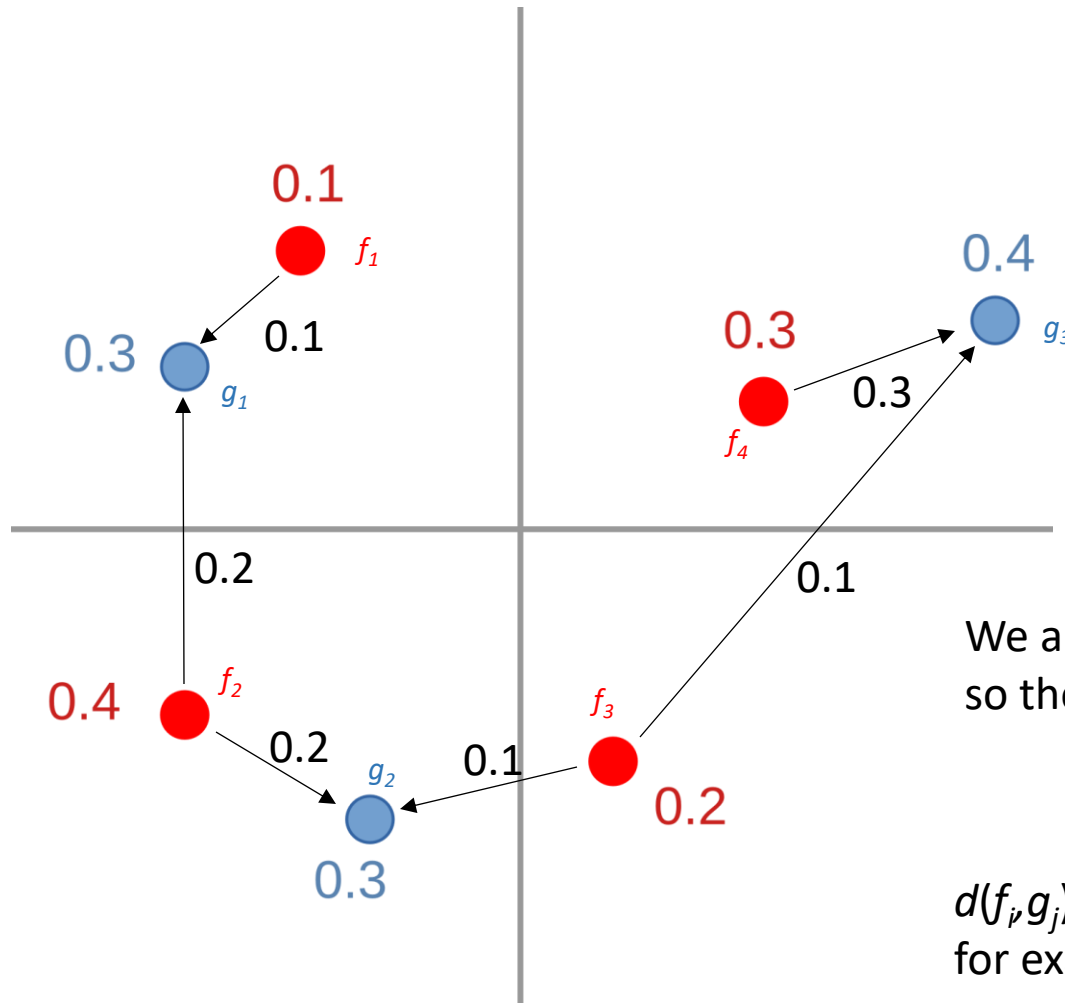
- Two distributions, f and g . We want to move f to g

Our dirt transport strategy

	f_1	f_2	f_3	f_4
g_1	0.1	0.2	0	0
g_2	0	0.2	0.1	0
g_3	0	0	0.1	0.3

This is just one strategy!

Wasserstein example



- Two distributions, f and g . We want to move f to g

Our dirt transport plan

	$f1$	$f2$	$f3$	$f4$
$g1$	0.1	0.2	0	0
$g2$	0	0.2	0.1	0
$g3$	0	0	0.1	0.3

$= \gamma$

We also need to take into account the distance d the dirt is moved, so the total work becomes the amount of dirt times the distance

$$d_w = \min \left\{ \sum_{i,j} \gamma_{i,j} d(f_i, g_j) \right\}$$

$d(f_i, g_j)$ tells us how much it costs to transport dirt from f_i to g_j for example, we could have $d(f_i, g_j) = \|f_i - g_j\|_2^2$

Style GAN

Spend two minutes on this website, which gives you fake images created using StyleGAN2

<https://thispersondoesnotexist.com/>



GANs are NOT reconstructing data!

The new information is learned from other samples to make a new sample look real, not be real.

Upscaling



Inpainting



What can we trust?

- With GANs and generative models, you can make fake images and fake videos that look real.
- We have already seen misuse. E.g., deep fakes, where people are unwilling being put in unwanted situations

Fake Putin:

[https://upload.wikimedia.org/wikipedia/commons/transcoded/a/af/Vladimir Putin warning Americans on election interference and increasing political divide.webm/Vladimir Putin warning Americans on election interference and increasing political divide.webm.720p.vp9.webm](https://upload.wikimedia.org/wikipedia/commons/transcoded/a/af/Vladimir_Putin_warning_Americans_on_election_interference_and_increasing_political_divide.webm/Vladimir_Putin_warning_Americans_on_election_interference_and_increasing_political_divide.webm.720p.vp9.webm)

Gentle introduction to deep fakes:

<https://www.kdnuggets.com/2018/03/exploring-deepfakes.html>



Fake presidents

Diffusion models

AU Deep Learning

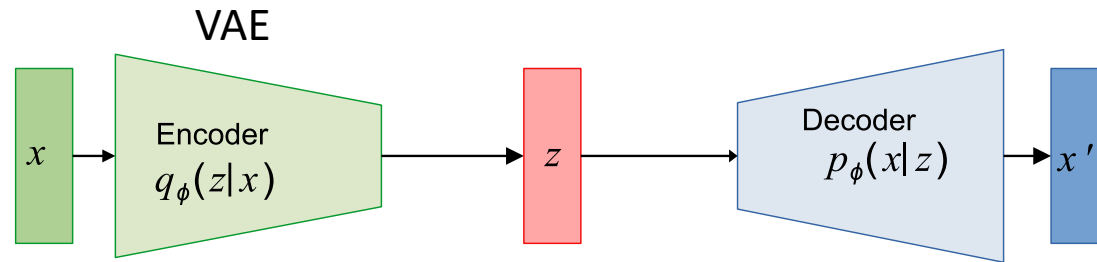
Papers to read

- Original from Jonathan Ho, Ajay Jain, and Pieter Abbeel
 - *Denoising Diffusion Probabilistic Models*
 - <https://arxiv.org/abs/2006.11239>
- Follow up with different tricks to improve the quality of generated images from Prafulla Dhariwal, and Alex Nichol
 - *Diffusion Models Beat GANs on Image Synthesis*
 - <https://arxiv.org/abs/2105.05233>
- Both are quite math heavy. Take a read, if you want all the details

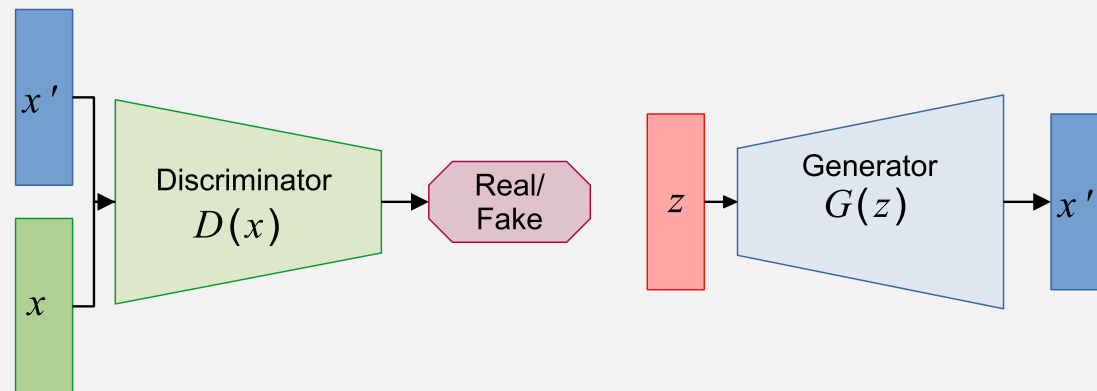


Sampling from noise

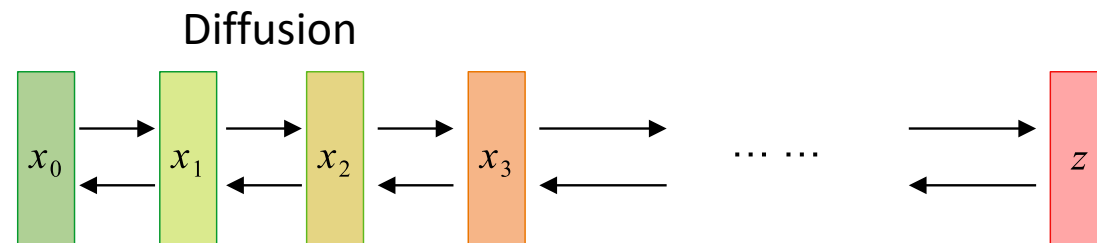
In both GAN and VAE we have only a single step from noise to image through a neural network. It makes the task very hard. In Diffusion models, we break this down into many small steps



VAE: Adversarial training

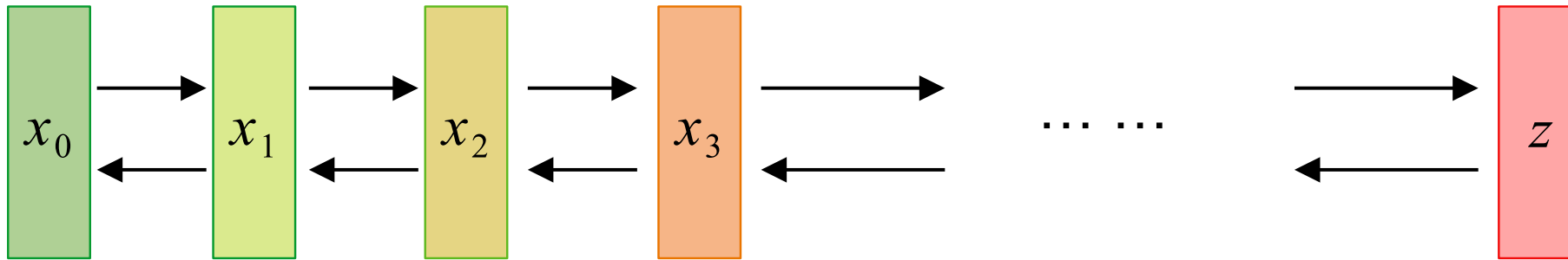


GAN: Adversarial training



Diffusion: Gradually add noise and then revert it

Diffusion models

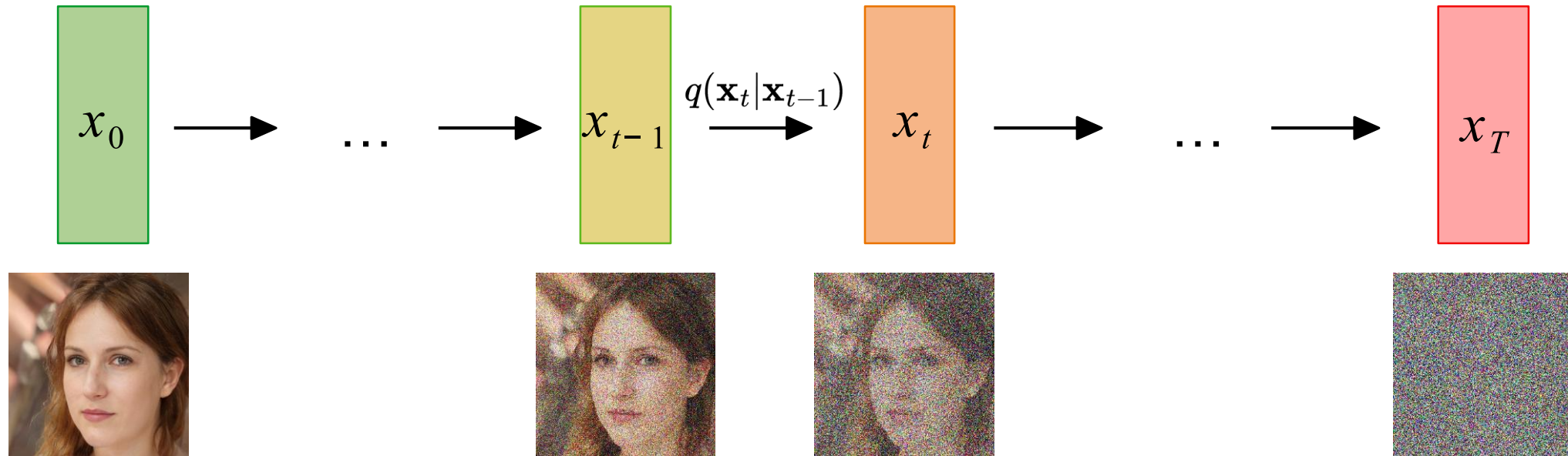


Convert a well-known *base distribution* (e.g. Gaussian) to the *target distribution* (image data) with small step sizes.

Estimating and analysing small step sizes is easier than describing a single step from noise to target distribution (like what VAE/GANs are doing)

Consider, for example, how one might model the distribution of all naturally-occurring photos. Each image is a point in the space of all images, and the distribution of naturally-occurring photos is a "cloud" in space. By repeatedly adding noise to the images, diffuses out to the rest of the image space, until the cloud becomes all but indistinguishable from a Gaussian distribution $N(0, I)$. A model that can approximately undo the diffusion can then be used to sample from the original distribution.

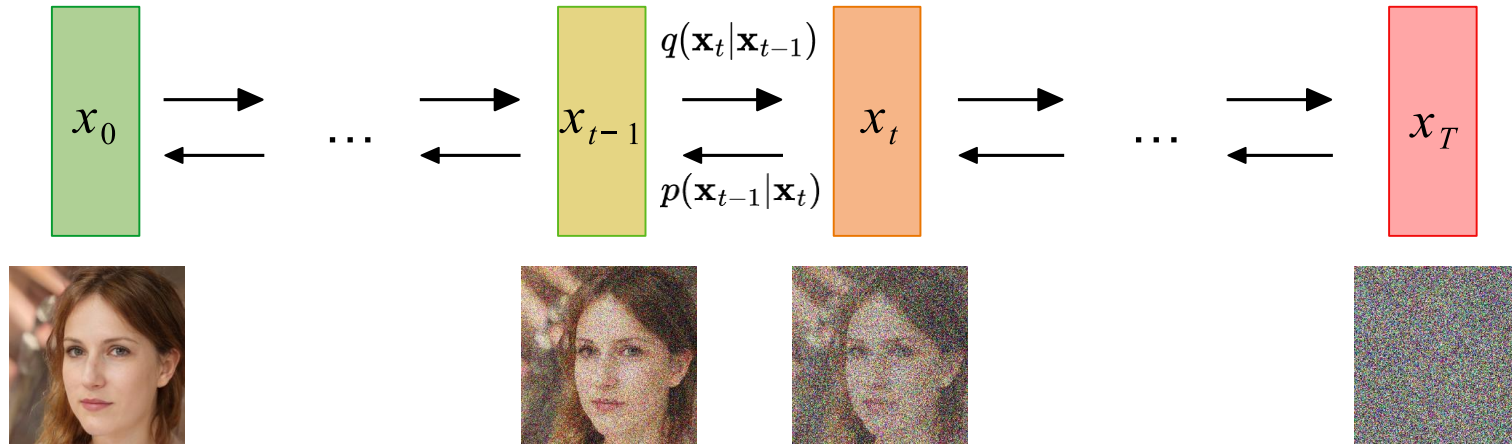
Forward process



- The **forward process** starts at a sample (image) from the real data distribution $x_0 \sim q$, where q is the probability distribution to be learned. Then repeatedly add add gaussian noise.
- The amount of noise added at each step is controlled by a noise schedule β . Usually, we can afford larger update steps when the sample gets noisier. For example^[3], a linear schedule from 0 to 1 might be used, or perhaps a geometric series.
- Repeat for T steps. When $T \rightarrow \infty$, x_T is equivalent to an isotropic Gaussian distribution.

where $z_1, \dots, z_T$ are IID samples from $N(0, I)$.

Backward diffusion



- The Goal is to reverse the diffusion process, $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$
- However, $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ is unknown!
- Therefore, we will instead learn p , that iteratively undo the forward process q .

$$\mathbf{x}_{t-1} \approx \mathbf{x}_t - \text{noise}$$

- We will learn to predict the noise mean

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)) \quad \leftarrow \text{Forward process}$$

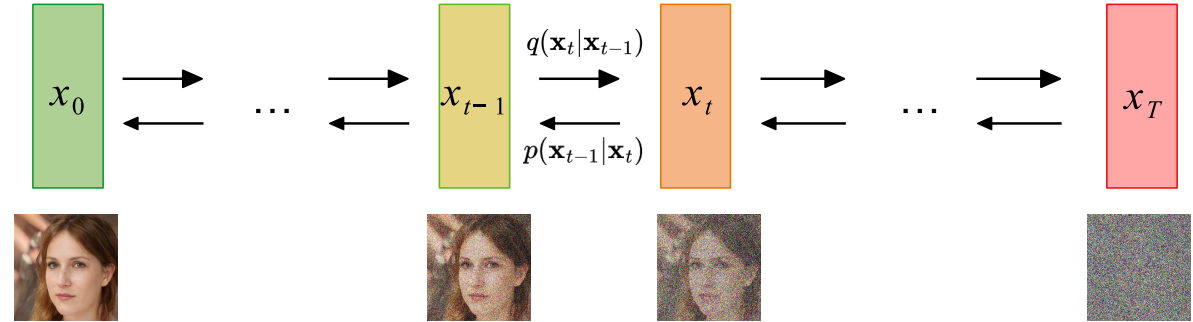
$$p_\theta(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) \quad \leftarrow \text{Reverse process}$$

The noise covariance will not contribute as much to the model as the noise mean. Therefore, the authors argue, that it can be fixed

$$\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t) = \sigma_t^2 \mathbb{I}$$

$$\sigma_t^2 = \beta_t$$

Training – Loss function



- Lots of math, KL-divergence, and properties of gaussian processes leads to the following loss function:

$$L_{t-1} = D_{KL}(q(x_{t-1} | x_t, x_0) || p_{\theta}(x_{t-1} | x_t))$$

- That can be converted to

$$L_{t-1} \propto ||\tilde{\mu}_t(x_t, x_0) - \mu_{\theta}(x_t, t)||^2$$

- Empirically, Ho *et al.* found that training μ_{θ} to predict the *noise* component at any given timestep yields better results. The final step involves rewriting this in terms of just the noise ϵ

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) \quad \text{where} \quad \alpha_t := 1 - \beta_t \quad \text{and} \quad \bar{\alpha}_t := \prod_{s=1}^t \alpha_s$$

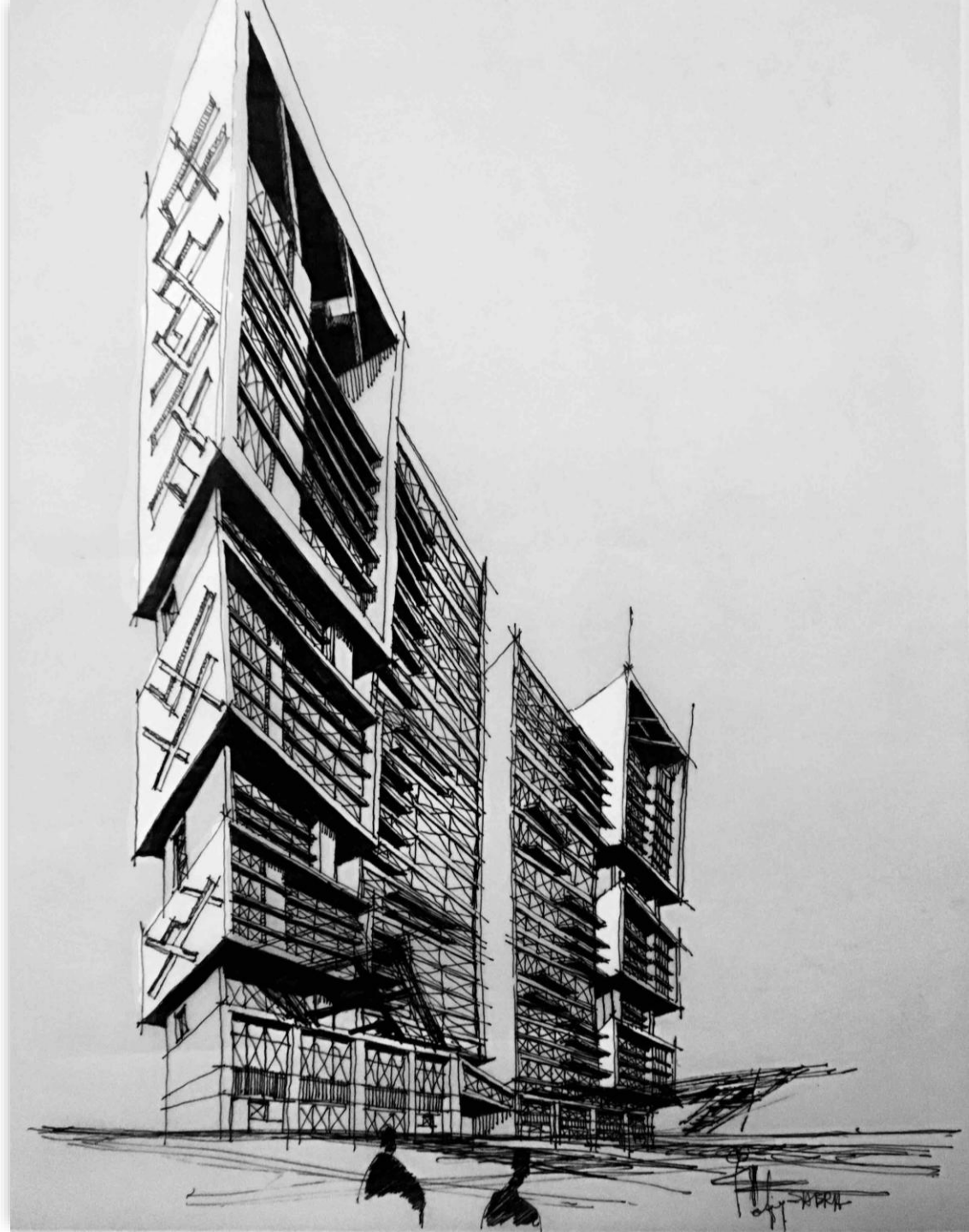
$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon,$$

- This leads to the following alternative loss function

$$L_{t-1} = \mathbb{E}_{x_0, \epsilon} \left[\frac{\beta_t}{2\sigma_t \alpha_t (1 - \bar{\alpha}_t)} |\epsilon - \epsilon_{\theta}(x_t, t)| \right]$$

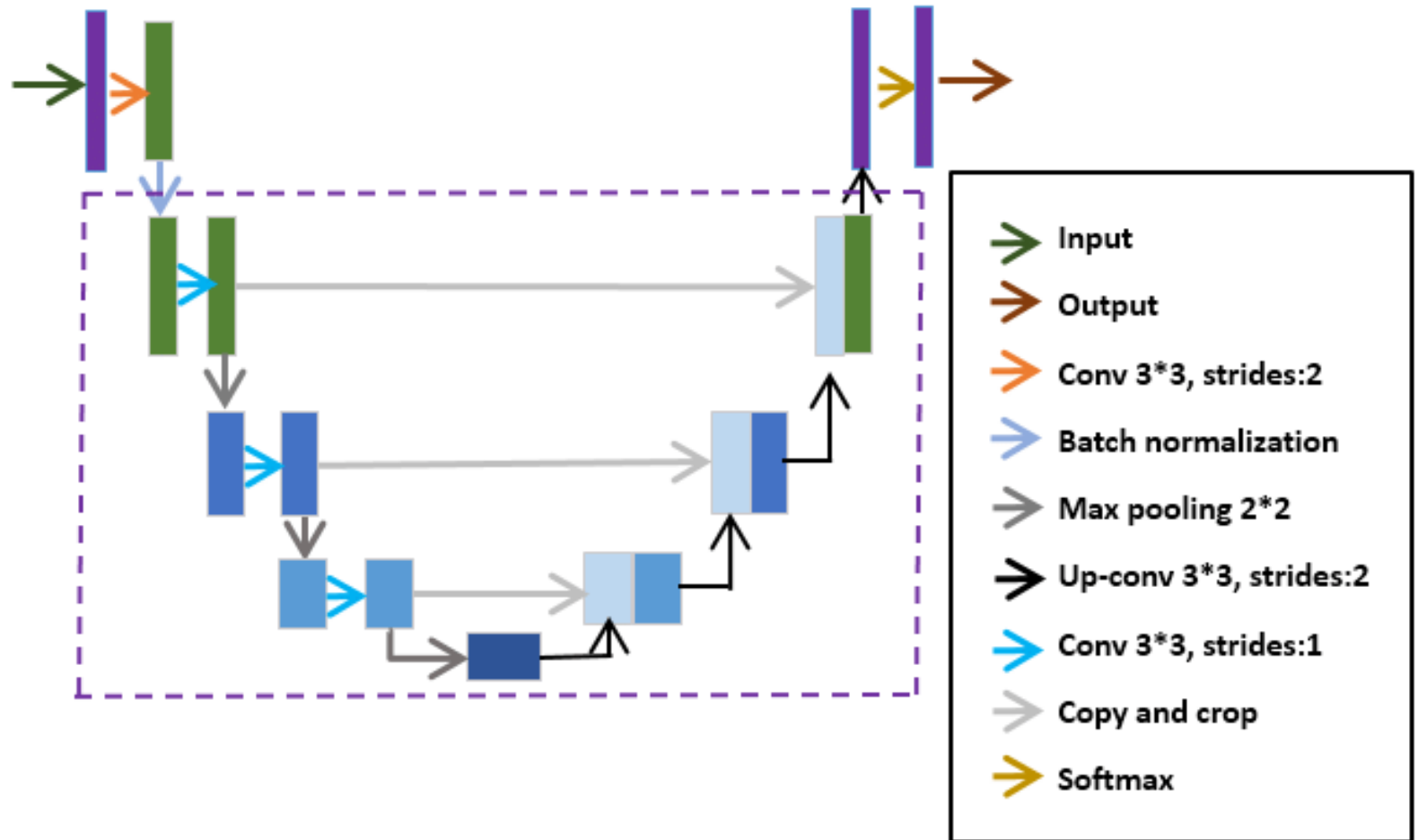
Network architecture

- We will use a neural network as the reverse process that undo the noise
- the *only* requirement for the model is that its input and output dimensionality are identical.
- A common choice is the UNET architecture



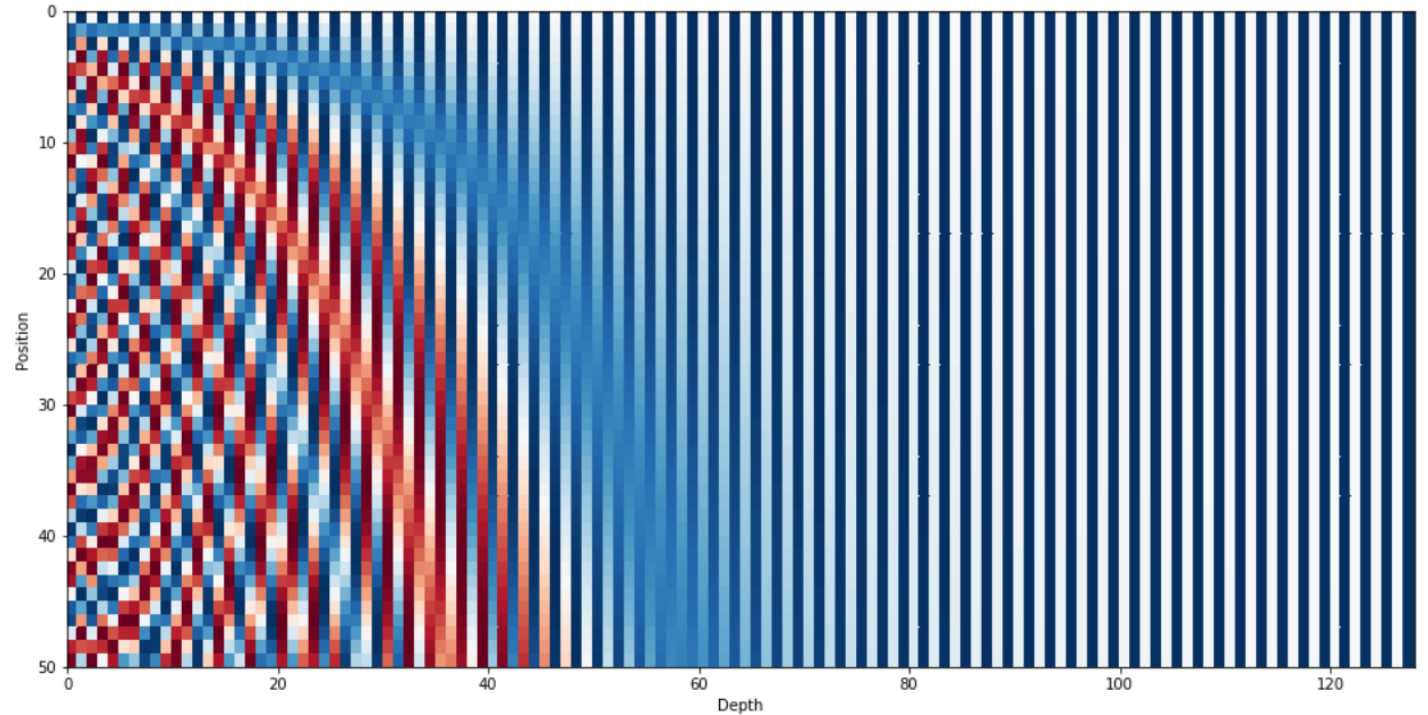
UNET

- Unlike autoencoders, there is no real encoder-decoder part
- In the original Diffusion models paper they added lots of modern image processing tricks: convnext, Group Norm, ResNet blocks, GeLU, ...
- The network will take in a noisy image at time t and output the noise added from time $1, \dots, t$
- I.e.: The network outputs the predicted noise, not the noise-free image



Time embedding

- We need to know at which time-step we are in order to know how much noise to remove.
- Therefore we typically add a positional embedding to encapture the position.
- The positional embedding is just added to the image together with the noise
- With the time embedding we can sample an image at a random time-step t and we will train **the network to predict all the noise added up to this time step**



Libraries

- Diffusion models can cost millions of kroner to train, but the model can run on normal laptops
- Stable Diffusion can be used for free:
 - <https://github.com/huggingface/diffusers>
- There exists tons of variations to stable diffusion to guide its output. E.g. pix2pix or QR-code generation



Exercise

- Use the MNIST data and make a simple MLP autoencoder with the following layer where the bottleneck size is 2. The input size will be $28 \times 28 = 784$ neurons at the input.

Layer type	Input size	Output size
linear + relu	# of pixels	100
linear	100	bottleneck_size
linear + relu	bottleneck_size	100
linear	100	# of pixels

- Show the encoded dimension for the test data set.
 - Can you choose some random numbers as encoded dimension and see what output MLP autoencoder generates?
- Modify the same network to make a variational autoencoder. Also show the encoded dimension and choose some numbers for encoded dimension yourself and see what output it generates. Feel free to get help from the internet 😊
- Read, understand and run the [DCGAN tutorial](#).